

Toolkit Documentation

Complete professional reference for every tool — with screenshots, technical details, and usage tips.

35

Tools

6

Categories

38

Screenshots

v1.0

Version

Table of Contents

All 35 tools listed by category with their routes.

CSV

Split CSV	/split
Merge Files	/merge
Convert CSV	/csv-convert

PDF — ORGANISE

Merge PDF	/pdf/merge
Split PDF	/pdf/split
Rotate Pages	/pdf/rotate
Delete Pages	/pdf/delete-pages
Extract Pages	/pdf/extract-pages
Reorder Pages	/pdf/reorder

PDF — ENHANCE

Compress PDF	/pdf/compress
Repair PDF	/pdf/repair
Watermark PDF	/pdf/watermark
Number Pages	/pdf/number-pages
Header & Footer	/pdf/header-footer
Sign PDF	/pdf/sign

PDF — CONVERT

PDF → Word / Excel	/pdf/conversion
PDF → PowerPoint	/pdf/to-pptx
Office → PDF	/pdf/office-to-pdf
PDF to Images	/pdf/to-images

PDF — OTHER

OCR	/pdf/ocr
Security	/pdf/security
Edit PDF	/pdf/editor
Compare PDFs	/pdf/compare
AI Assistant	/pdf/ai

IMAGE

Convert Format	/image/convert
Compress Image	/image/compress
Resize Image	/image/resize
Rotate & Flip	/image/rotate
Images → PDF	/image/to-pdf
Crop Image	/image/crop
Watermark Image	/image/watermark
Image Collage	/image/collage

RESUME & DOCUMENTS

Resume Maker	/resume/
Cover Letter Maker	/cover-letter/
Invoice Generator	/invoice/

Architecture Overview

Technology stack, key libraries, and design decisions behind the Toolkit.

BACKEND

Flask 3.x + Python 3.11

Synchronous WSGI app. Each tool category is a Flask Blueprint with its own `url_prefix`. No database — all processing is stateless.

FRONTEND

Tailwind CSS + Vanilla JS

No frontend framework. Jinja2 templates with Tailwind CDN, Font Awesome icons, and plain `fetch()` for async file operations.

PDF PROCESSING

pikepdf + PyMuPDF + WeasyPrint

pikepdf for manipulation (merge, split, encrypt), PyMuPDF (fitz) for rendering pages to images, WeasyPrint for HTML→PDF generation.

IMAGE PROCESSING

Pillow (PIL)

All image operations use Pillow. All output files use UUID-prefixed names to prevent stale-file accumulation across runs.

DATA PROCESSING

pandas + openpyxl

CSV/TSV reading and merging via pandas. Excel export uses the openpyxl engine. BytesIO streaming avoids intermediate disk writes.

AI

Ollama (local LLM)

All AI features use the Ollama REST API at `localhost:11434`. Fully local inference — no data is sent to any cloud service.

PRIVACY

Local-first, no retention

Files are processed on-server and served back immediately. No user data is retained after the HTTP response is delivered.

BUG-FREE OUTPUT

UUID filenames + rmtree

All task outputs use UUID-prefixed filenames. The CSV split directory is wiped with `shutil.rmtree` before each run to prevent stale parts from prior runs accumulating in the ZIP.

CSV

CSV

/split

Split CSV

Splits a large CSV file into equal-row chunks and returns all parts bundled in a ZIP archive. Useful for breaking datasets that exceed row limits in spreadsheet applications.

KEY FEATURES

- ✓ Configurable rows per chunk — default 500, minimum 10
- ✓ Header row automatically included in every output file
- ✓ Leftover rows below minimum are merged into the last part
- ✓ Split directory wiped before each run — no stale file accumulation
- ✓ All parts returned in a single ZIP for one-click download
- ✓ Processes files entirely server-side using Python's csv module
- ✓ Flash message reports total parts and row count after split
- ✓ Supports any valid UTF-8 encoded CSV file

HOW IT WORKS

Python's csv module reads the file row-by-row. Before writing, shutil.rmtree clears the split directory named

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	CSV file	Required	The CSV file to split
row_count	Integer	Optional	Rows per output file (default: 500, min: 10)

OUTPUT

ZIP archive containing N CSV files named {stem}_part_1.csv through {stem}_part_N.csv, each with the original header row.

LIBRARIES

csv (stdlib)

zipfile

shutil

Flask

after the uploaded file, preventing stale output from a previous run with a different row count. Part files are written sequentially; rows below `MIN_ROWS_PER_FILE` are appended to the last part. All parts are zipped and returned.

USAGE TIPS

1. Always inspect the last part — it may have fewer rows than the others.
2. Use a row count that divides your data evenly to get uniform parts.
3. Re-upload the same file with a different row count — stale files are cleared automatically.
4. Large files (1M+ rows) may take several seconds to split.
5. The output ZIP preserves the original filename stem for easy identification.

EDITOR'S NOTE

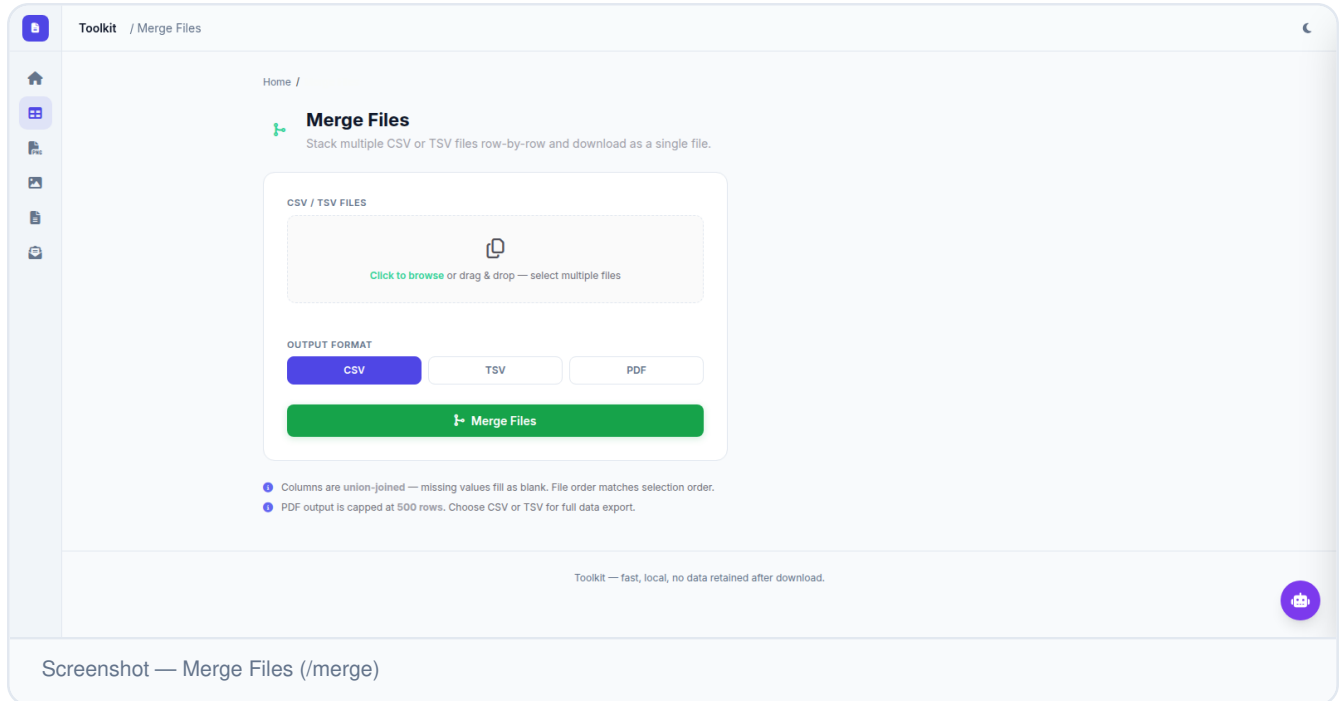
Splitting oversized CSV files into uniform row-based chunks is a common preprocessing step before bulk imports, parallel pipelines, or API batch submissions. This tool accepts a CSV and a target chunk size, divides the file into equal-row segments, and returns all chunks as a single ZIP archive for convenient download. A key design decision is the use of ``shutil.rmtree`` to unconditionally purge any previously generated output directory before each run, preventing stale chunks from earlier sessions from contaminating the current result. Practitioners should ensure the input file uses consistent delimiters and that the header row is present, as each output chunk preserves the original header automatically.

CSV

/merge

Merge Files

Merges multiple CSV or TSV files into a single consolidated file using a column union strategy. Missing columns are filled with empty values.



Screenshot — Merge Files (/merge)

KEY FEATURES

- ✓ Accepts any number of CSV and TSV files simultaneously
- ✓ Auto-detects separator: tab for .tsv, comma for .csv
- ✓ Column union — all columns from all files appear in the output
- ✓ Missing column values filled with empty string
- ✓ Output formats: CSV, TSV, or PDF summary table
- ✓ Rows concatenated in upload order with a reset index
- ✓ Handles inconsistent schemas across files gracefully
- ✓ Uses pandas for robust parsing and encoding handling

HOW IT WORKS

Each uploaded file is saved and read with `pandas.read_csv` using separator auto-detection. DataFrames are concatenated with `pd.concat(axis=0, ignore_index=True, sort=False)`, which takes the union

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>files[]</code>	Multiple files	Required	Two or more CSV or TSV files
<code>format</code>	csv tsv pdf	Optional	Output format (default: csv)

OUTPUT

Single merged file (`merged_output.csv`, `.tsv`, or `.pdf`) with all rows from all inputs.

LIBRARIES

pandas

fpdf

Flask

USAGE TIPS

1. Upload files in the order you want rows to appear.
2. Column names are case-sensitive — ensure consistency across files.

of all columns. Missing values become NaN displayed as empty cells. Output is written to MERGE_FOLDER as merged_output.{format}.

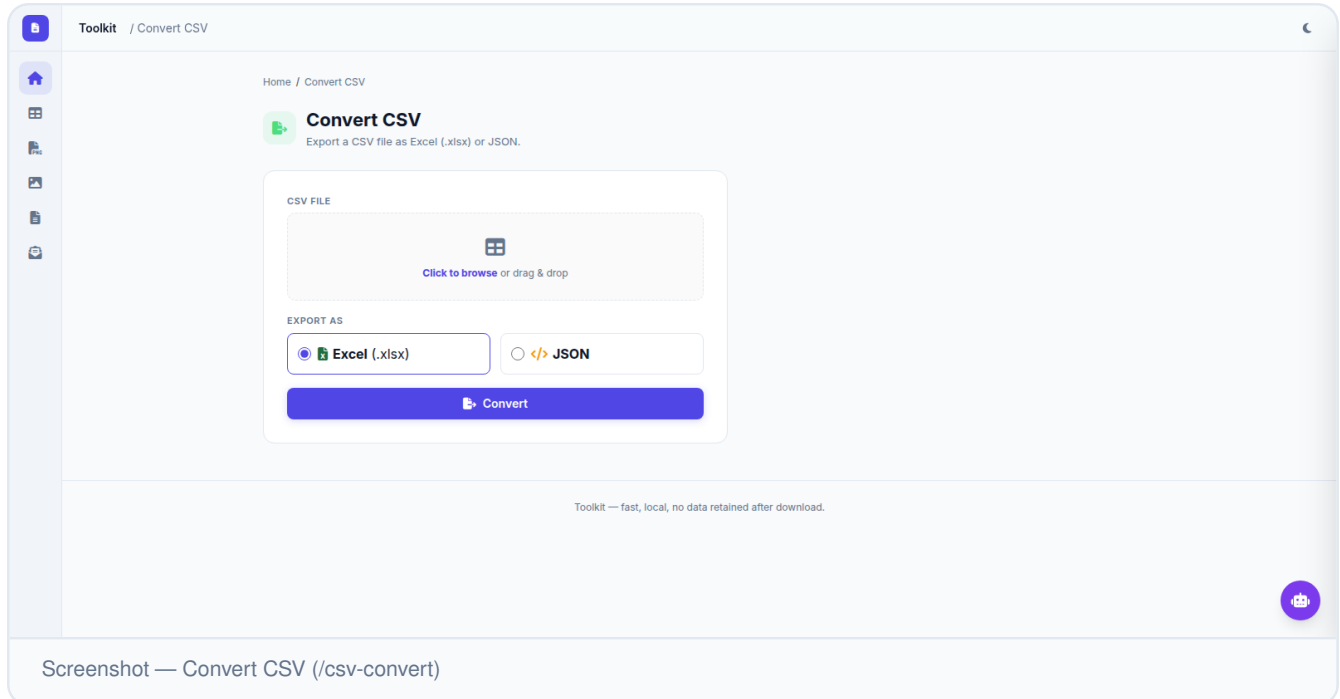
3. For large merges, CSV output is fastest; PDF is slowest.
4. Inspect the merged file header to confirm column union worked correctly.
5. TSV output is useful when values contain commas.

EDITOR'S NOTE

Designed for analysts and data engineers who routinely consolidate fragmented datasets, this tool accepts multiple CSV or TSV files and produces a single unified output via pandas column-union concatenation. Rather than requiring every input file to share an identical schema, the column-union strategy takes the superset of all column headers, filling gaps with NaN where a given file lacks a particular field — a deliberate trade-off that prioritizes flexibility over strict schema enforcement. This behavior makes it well-suited for heterogeneous exports from the same source system over time, but consumers should be prepared to handle or impute null values downstream. Files with inconsistent delimiter usage or mixed encodings should be normalized before merging to avoid silent data corruption.

Convert CSV

Exports a CSV file as Excel (.xlsx) or JSON with a single click. No intermediate file is written to disk — results stream directly to the browser.



Screenshot — Convert CSV (/csv-convert)

KEY FEATURES

- ✓ Convert CSV to Excel (.xlsx) with full header row preserved
- ✓ Convert CSV to JSON array-of-records with 2-space indentation
- ✓ No intermediate file written — streams BytesIO directly
- ✓ Correct MIME types set for both output formats
- ✓ Download filename matches the original CSV stem
- ✓ Uses openpyxl engine for proper .xlsx formatting
- ✓ JSON output uses orient="records" for maximum compatibility
- ✓ Two-button format selector with active-state highlighting

HOW IT WORKS

pandas.read_csv reads the uploaded stream. For Excel: df.to_excel(BytesIO(), engine="openpyxl") writes bytes without touching disk; the buffer is served with the correct xlsx MIME type. For JSON:

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	CSV file	Required	The CSV file to convert
target	excel json	Required	Target output format

OUTPUT

Excel file (.xlsx) or JSON file (.json) with the same stem as the uploaded CSV.

LIBRARIES

pandas

openpyxl

io.BytesIO

Flask

USAGE TIPS

1. Excel output preserves column order from the original CSV.

`df.to_json(orient="records", indent=2)` returns a Flask Response with Content-Disposition forcing a download.

2. JSON records format is directly consumable by most REST APIs and JavaScript apps.
3. Large CSVs convert quickly — pandas processes in-memory.
4. Column names become Excel headers automatically.
5. Use JSON output when feeding data to another application or service.

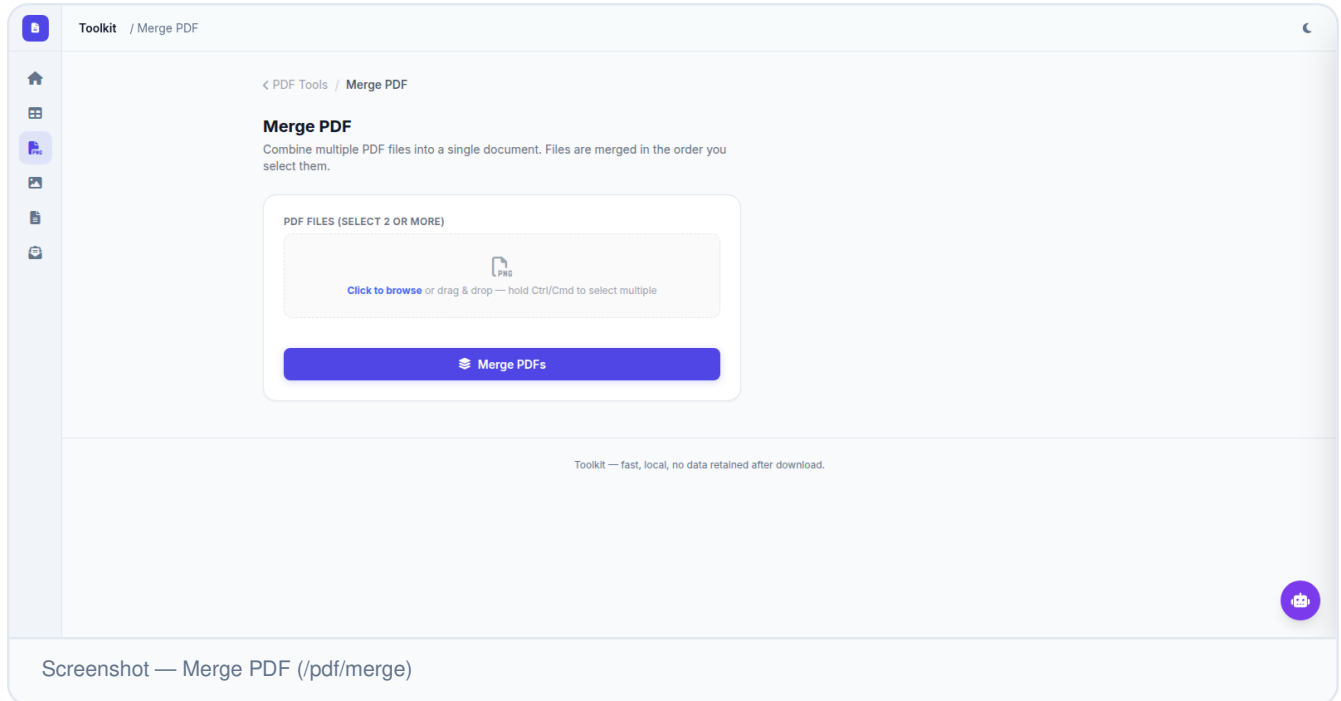
EDITOR'S NOTE

Designed for lightweight data transformation workflows, the Convert CSV tool accepts a comma-separated file and produces either an Excel workbook or a JSON document without touching the filesystem. Internally, pandas handles parsing and conversion while a BytesIO buffer streams the result directly to the HTTP response, eliminating temporary file management and reducing I/O overhead. This in-memory approach also improves security by avoiding residual data on disk between requests. When targeting Excel output, the tool writes an .xlsx file via openpyxl; JSON output follows a records-oriented structure suitable for downstream API consumption. Input files with inconsistent encodings or malformed delimiters should be normalized before submission to avoid silent data loss.

PDF — Organise

Merge PDF

Combines multiple PDF files into a single document in the order they were uploaded. Preserves all pages, content, and formatting from every input file.



KEY FEATURES

- ✓ Accepts any number of PDF files in one upload
- ✓ Pages appended strictly in upload order
- ✓ Preserves original page dimensions, content, and annotations
- ✓ Handles PDF versions 1.0 through 2.0
- ✓ UUID-named output prevents filename collisions
- ✓ Returns merged PDF immediately as an attachment
- ✓ Efficient streaming — pikepdf appends pages without full re-render
- ✓ Works with password-free PDFs of any size

HOW IT WORKS

Each uploaded file is saved to `UPLOAD_FOLDER`. A blank `pikepdf.Pdf` is created. For each source PDF, `pikepdf.Pdf.open()` loads the file and `out.pages.extend(src.pages)` appends all its pages to the output. The result is saved to a UUID-named path and returned with Flask `send_file`.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>files[]</code>	PDF files	Required	Two or more PDF files to merge

OUTPUT

Single merged PDF containing all pages from all input files in upload order.

LIBRARIES

pikepdf

Flask

werkzeug

USAGE TIPS

1. Drag files in the desired order before uploading.
2. There is no practical limit on the number of files.
3. Password-protected PDFs must be unlocked before merging.
4. Resulting file size is approximately the sum of all inputs.

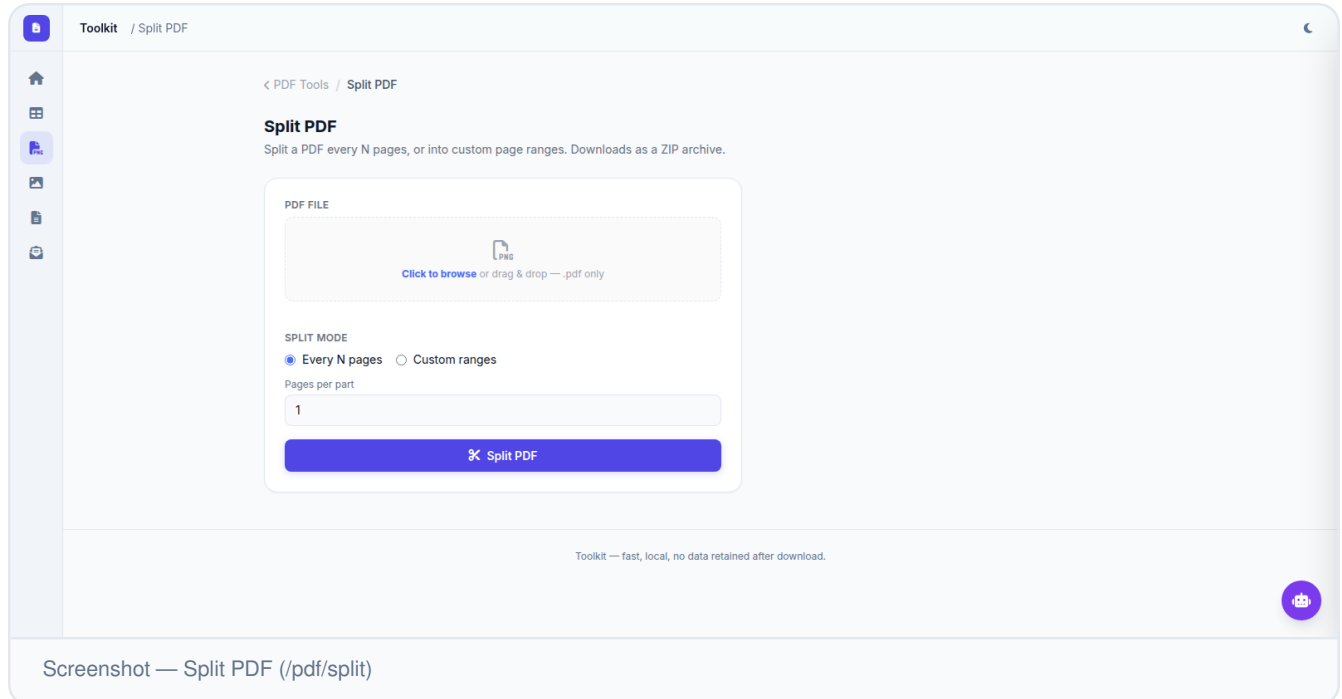
5. Use Reorder Pages after merging if you need to fine-tune page sequence.

EDITOR'S NOTE

Combining multiple PDF documents into a single file, the Merge PDF tool leverages pikepdf's low-level page-appending mechanism to assemble source PDFs in caller-specified order without re-encoding content. Each output file receives a UUID-based filename, eliminating collisions in shared or concurrent environments. Because pikepdf operates directly on the PDF object graph, existing bookmarks, annotations, and embedded metadata from individual source files are preserved at the page level, though document-level metadata from later files will overwrite earlier entries. Callers should ensure all source PDFs are accessible and not password-protected before invoking the tool; encrypted inputs will raise an exception rather than silently produce a corrupt output.

Split PDF

Splits a PDF into multiple files by page count or by custom page ranges. Returns all parts as a single ZIP archive.



KEY FEATURES

- ✓ Mode 1: split every N pages (e.g. every 2 pages)
- ✓ Mode 2: split by custom ranges (e.g. "1-3,5,7-9")
- ✓ Range parser handles mixed single pages and ranges
- ✓ Each output part uses a UUID prefix to avoid conflicts
- ✓ All parts returned in a single ZIP archive
- ✓ 1-indexed page numbers match user expectations
- ✓ Zero-indexed internally for accurate pikepdf access
- ✓ Preserves all page content and metadata in each part

HOW IT WORKS

pikepdf opens the source PDF and counts total pages. In "every" mode, chunks are computed as $[(0, n-1), (n, 2n-1) \dots]$. In "ranges" mode, the `_parse_ranges()` function tokenises the input string handling "N-M" and single "N" entries, converting 1-indexed to 0-indexed.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	PDF file	Required	The PDF to split
<code>mode</code>	every ranges	Required	Split mode
<code>every</code>	Integer	Conditional	Pages per part (mode=every)
<code>ranges</code>	String	Conditional	e.g. "1-3,5,7-9" (mode=ranges)

OUTPUT

ZIP archive of PDF parts named `part_1.pdf`, `part_2.pdf`, etc.

LIBRARIES

pikepdf

zipfile

Flask

Each chunk is written to a UUID-named PDF and added to a ZIP.

USAGE TIPS

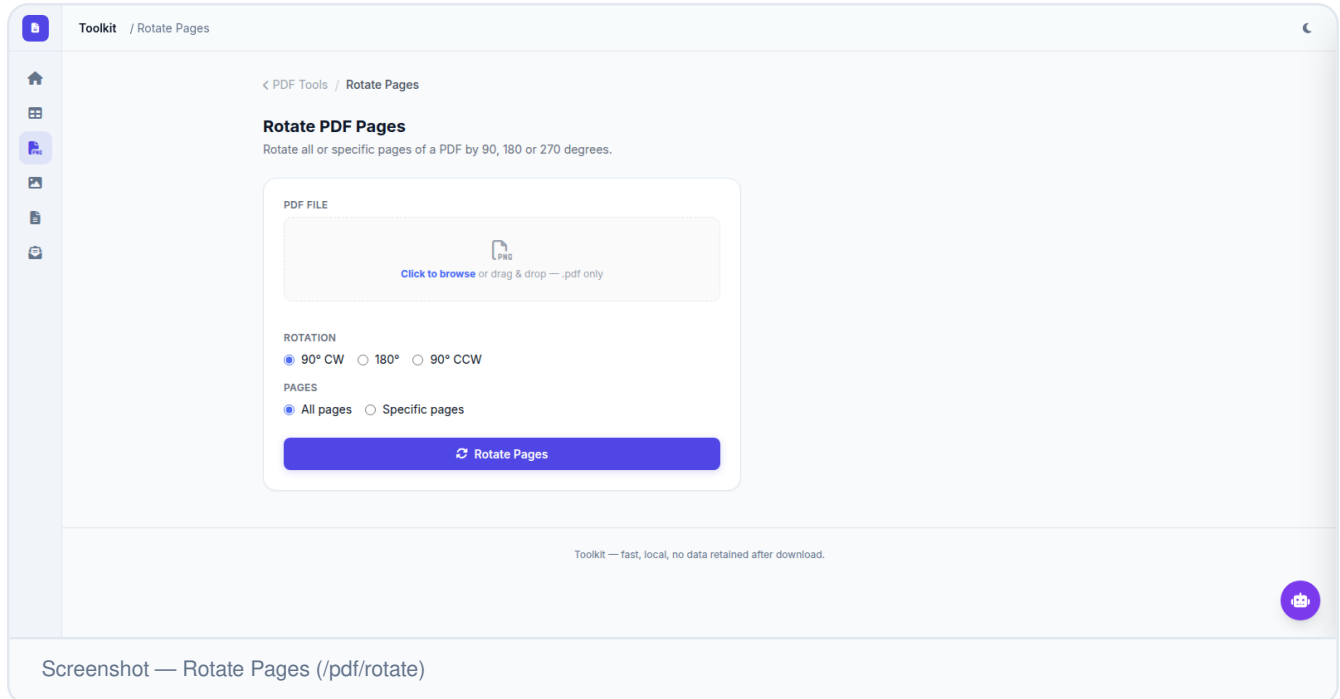
1. Use "every 1" to extract every page as a separate file.
2. Ranges like "1-3,5,8-10" give you full control over grouping.
3. Page numbers in ranges are 1-indexed.
4. Resulting ZIP size equals the original PDF size approximately.
5. Combine with Merge PDF to reorganise a document by reordering split parts.

EDITOR'S NOTE

Splitting a PDF into smaller, independently distributable files is a common workflow requirement across legal, publishing, and archival contexts. This tool accepts either a uniform page stride — every N pages — or a comma-separated range expression such as "1-3,5,7", enabling both bulk segmentation and surgical extraction in a single operation. Each invocation produces a UUID-named ZIP archive, which prevents filename collisions in multi-user or automated pipeline environments and makes output artifacts safe to store in shared object storage without manual renaming. Ranges are validated before any file I/O begins, so malformed expressions fail fast rather than producing partial output. Source documents with password protection must be decrypted prior to submission.

Rotate Pages

Rotates all or specific pages of a PDF by 90, 180, or 270 degrees. Non-destructive — writes to a new output file.



Screenshot — Rotate Pages (/pdf/rotate)

KEY FEATURES

- ✓ Rotation angles: 90°, 180°, or 270° clockwise
- ✓ Target all pages or a comma-separated list of page numbers
- ✓ 1-indexed page numbers for intuitive user input
- ✓ Handles PDFs with pre-existing /Rotate entries correctly
- ✓ Rotation added to existing angle: $(\text{old} + \text{new}) \% 360$
- ✓ UUID output filename prevents cache issues
- ✓ Original PDF is never modified
- ✓ Returns rotated PDF immediately

HOW IT WORKS

pikepdf opens the PDF. If pages="all", every page is iterated; otherwise the comma-separated list is parsed and converted to 0-indexed. For each target page, the /Rotate key in the page dictionary is updated with $(\text{existing_rotation} + \text{angle}) \% 360$. pikepdf normalises rotation values. Saves to UUID output.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to rotate
angle	90 180 270	Required	Degrees clockwise
pages	all or "1,3,5"	Optional	Pages to rotate (default: all)

OUTPUT

PDF with specified pages rotated to the new angle.

LIBRARIES

pikepdf

Flask

USAGE TIPS

1. Rotate only specific pages by entering them as "1,3,5".
2. Rotating 180° is useful for upside-down scanned pages.

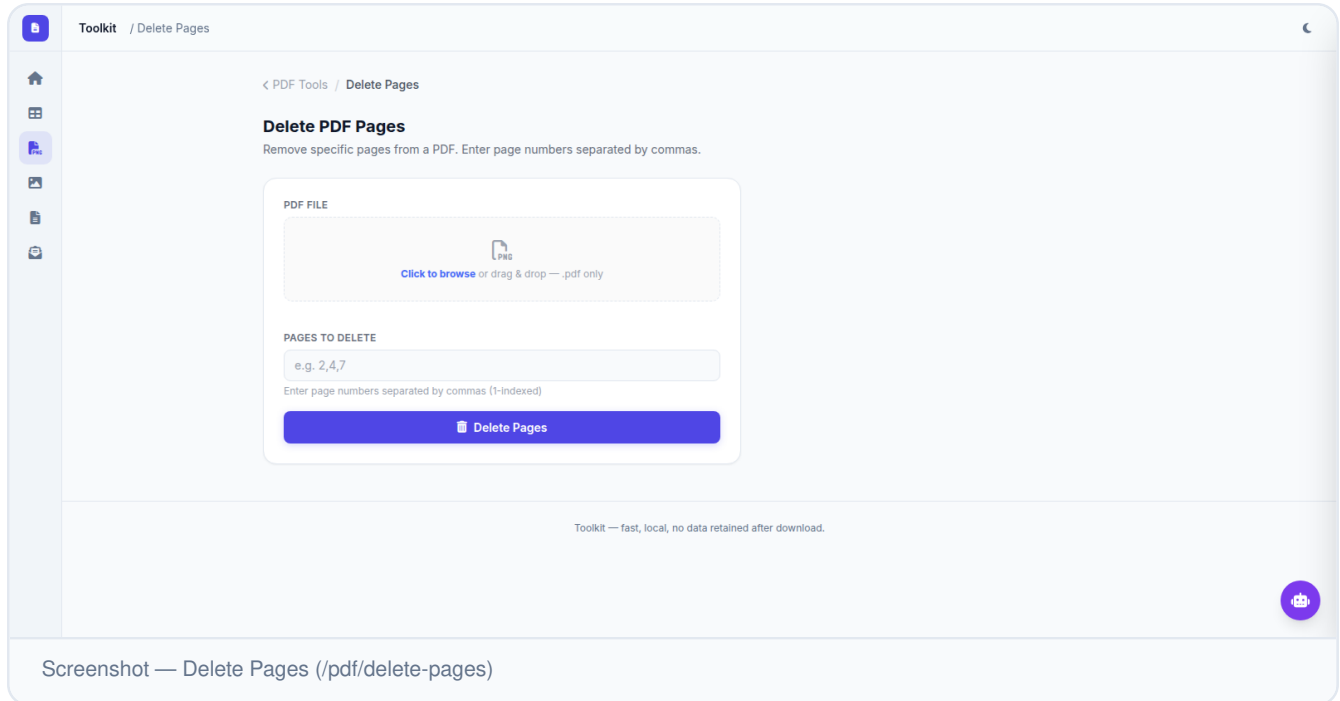
3. Apply 270° for pages scanned in landscape that should be portrait.
4. Run the tool twice to accumulate rotation (e.g. 90°+90° = 180°).
5. Check the result in a PDF viewer before distributing.

EDITOR'S NOTE

Page rotation in PDF documents is handled by updating the `/Rotate`` key in each affected page's dictionary rather than physically rewriting the page content stream. This approach, implemented via `pikepdf`, is non-destructive: the underlying graphic content remains untouched, and rotation values are stored as metadata the viewer interprets at render time. Supported angles — 90, 180, and 270 degrees — are applied cumulatively to any existing rotation, so callers should inspect the current `/Rotate`` value before invoking the tool to avoid unintended compounding. Output files preserve all other page attributes, annotations, and embedded resources. Best practice is to operate on a copy of the source PDF when batch-rotating mixed-orientation documents.

Delete Pages

Permanently removes specified pages from a PDF. Pages are deleted in reverse order internally to preserve index integrity.



Screenshot — Delete Pages (/pdf/delete-pages)

KEY FEATURES

- ✓ Remove any combination of pages in one operation
- ✓ 1-indexed page numbers match user expectations
- ✓ Pages deleted in reverse order to preserve indices
- ✓ Prevents deletion of all pages — minimum 1 retained
- ✓ UUID output filename prevents cache collisions
- ✓ Original PDF is never modified
- ✓ Accepts comma-separated page list (e.g. "2,5,7")
- ✓ Returns modified PDF immediately

HOW IT WORKS

pikepdf opens the PDF. The user-supplied comma-separated page list is parsed, converted to 0-indexed integers, sorted in descending order, and deduplicated. Pages are deleted using `del pdf.pages[idx]` in reverse — this prevents index shifting that would occur with forward deletion. Saves to UUID output.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to modify
pages	String	Required	Comma-separated page numbers to delete, e.g. "2,5,7"

OUTPUT

PDF with the specified pages permanently removed.

LIBRARIES

pikepdf

Flask

USAGE TIPS

1. Double-check page numbers before deleting — this operation is irreversible.
2. Use Extract Pages instead if you want to keep a subset rather than remove a few.

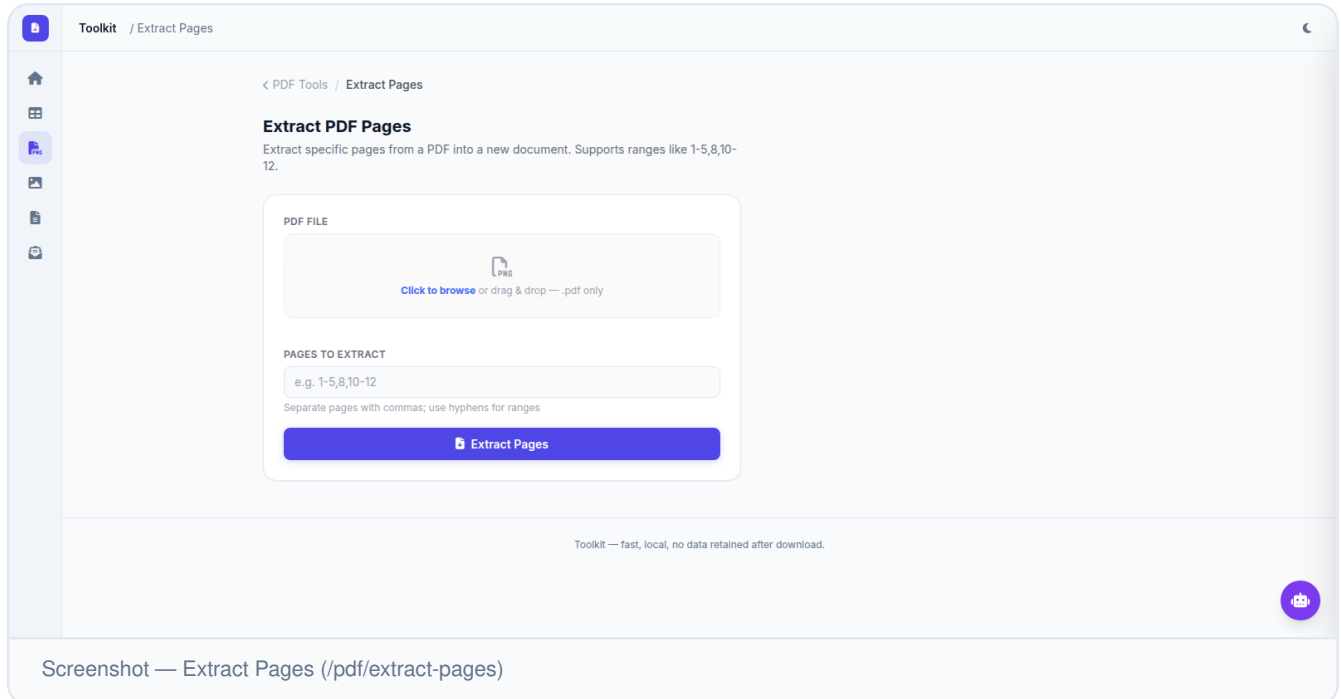
3. Preview the PDF page count to ensure you have the right page numbers.
4. Large page lists are processed in one pass — no need to delete one at a time.
5. The output file size decreases proportionally to the pages removed.

EDITOR'S NOTE

Removing specific pages from a PDF is a common document management task, and this tool handles it with a deliberate internal ordering strategy: deletions are applied in reverse index order so that removing earlier pages does not shift the positions of later ones, keeping all target indices valid throughout the operation. Users specify pages by number, and the tool resolves the correct removal sequence automatically. This approach eliminates the off-by-one errors that naive sequential deletion produces. One important caveat: page numbers are assumed to be one-based, matching conventional PDF viewer displays. For bulk operations, verifying the final page count against expectations is recommended before overwriting the source file.

Extract Pages

Saves a specific subset of pages from a PDF as a new standalone file. The original PDF is left untouched.



Screenshot — Extract Pages (/pdf/extract-pages)

KEY FEATURES

- ✓ Extract any subset of pages into a new PDF
- ✓ 1-indexed page numbers for intuitive input
- ✓ Supports comma-separated list and ranges like "1-3,5"
- ✓ Extracted pages maintain their original order
- ✓ Original PDF is never modified
- ✓ UUID output filename
- ✓ Useful for sharing chapters or sections independently
- ✓ Preserves all page content, annotations, and embedded fonts

HOW IT WORKS

pikepdf opens the source PDF. The page list is parsed with the same range parser as Split PDF. A new pikepdf.Pdf is created, and only the specified pages are appended using `new_pdf.pages.append(src.pages[idx])`. Saved to UUID output path and returned as download.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The source PDF
pages	String	Required	Pages to extract, e.g. "1,3-5,8"

OUTPUT

New PDF containing only the extracted pages in the specified order.

LIBRARIES

pikepdf

Flask

USAGE TIPS

1. Extract a single chapter by specifying its page range.
2. Combine with Merge PDF to assemble a new document from extracted sections.
3. Page numbers are 1-indexed — page 1 is the cover.

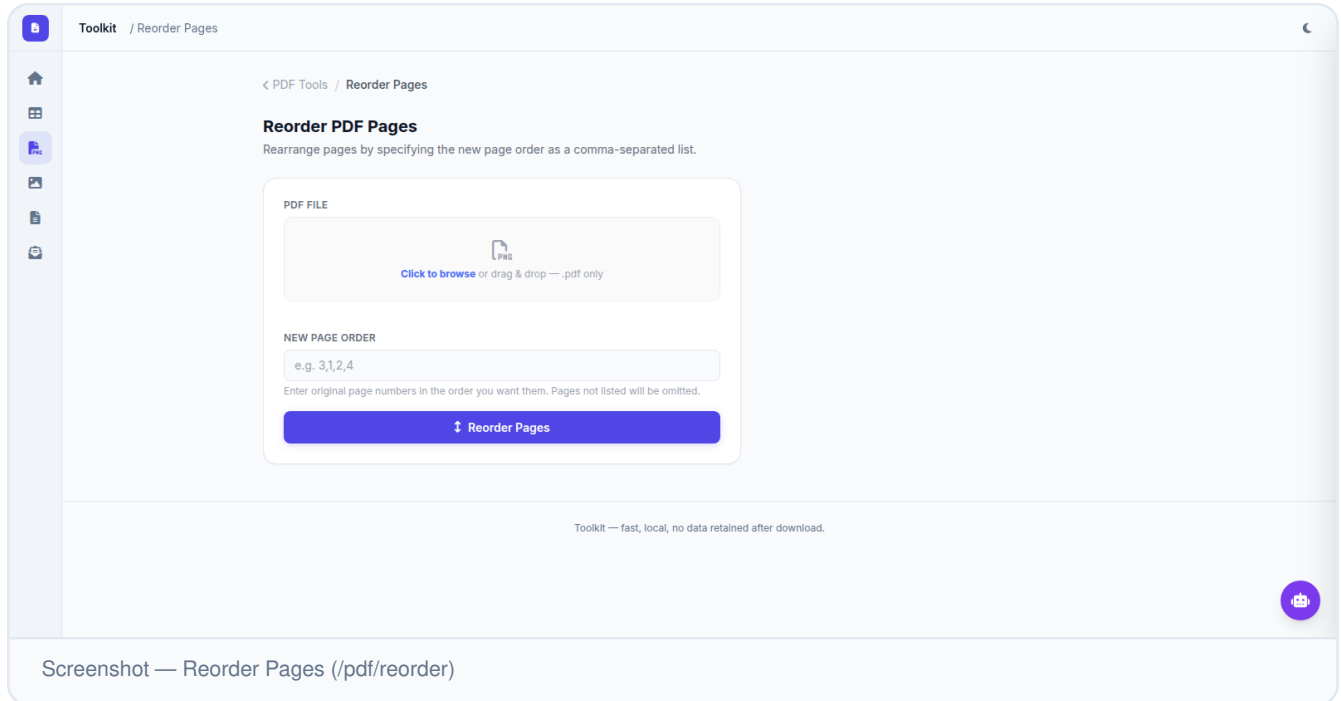
4. You can repeat a page number to duplicate it in the output.
5. The output is a fully independent PDF — not a reference to the original.

EDITOR'S NOTE

Extracting a page subset from a PDF produces a fully self-contained file: all referenced fonts, embedded images, and cross-references are resolved and included so the output opens correctly in any compliant viewer without dependency on the source document. This makes the tool well-suited for distributing chapters, isolating appendices, or archiving specific sections independently. Page selection uses a one-based index consistent with visible page numbers in most PDF readers, reducing off-by-one errors. One important consideration: interactive elements such as form fields, bookmarks, and internal hyperlinks that target pages outside the selected range are silently dropped, so users should verify interactive content after extraction rather than assuming full fidelity.

Reorder Pages

Rearranges PDF pages into any custom order specified by the user. Supports page duplication and omission.



Screenshot — Reorder Pages (/pdf/reorder)

KEY FEATURES

- ✓ Specify any permutation of page numbers
- ✓ Duplicate a page by repeating its number in the order
- ✓ Omit a page by simply not including its number
- ✓ New order entered as comma-separated 1-indexed numbers
- ✓ Resulting PDF has exactly as many pages as numbers supplied
- ✓ Original PDF is never modified
- ✓ UUID output filename
- ✓ Error returned if any page number is out of range

HOW IT WORKS

pikepdf opens the source PDF. The user-supplied order string is split and each element converted to a 0-indexed integer. A new PDF is built by appending `pdf.pages[idx]` for each value. This allows duplication (repeating a number) and omission (skipping a number). Saves to UUID output.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to reorder
order	String	Required	New page order e.g. "3,1,2,4"

OUTPUT

PDF with pages in the user-specified order.

LIBRARIES

pikepdf

Flask

USAGE TIPS

1. Enter all page numbers to reorder the whole document.
2. Omit a page number to create a version with that page removed.

3. Repeat a page number to create a duplicate of that page.
4. Use a spreadsheet to plan complex reorderings before entering them.
5. Combine with Split PDF to reorder sections of a large document.

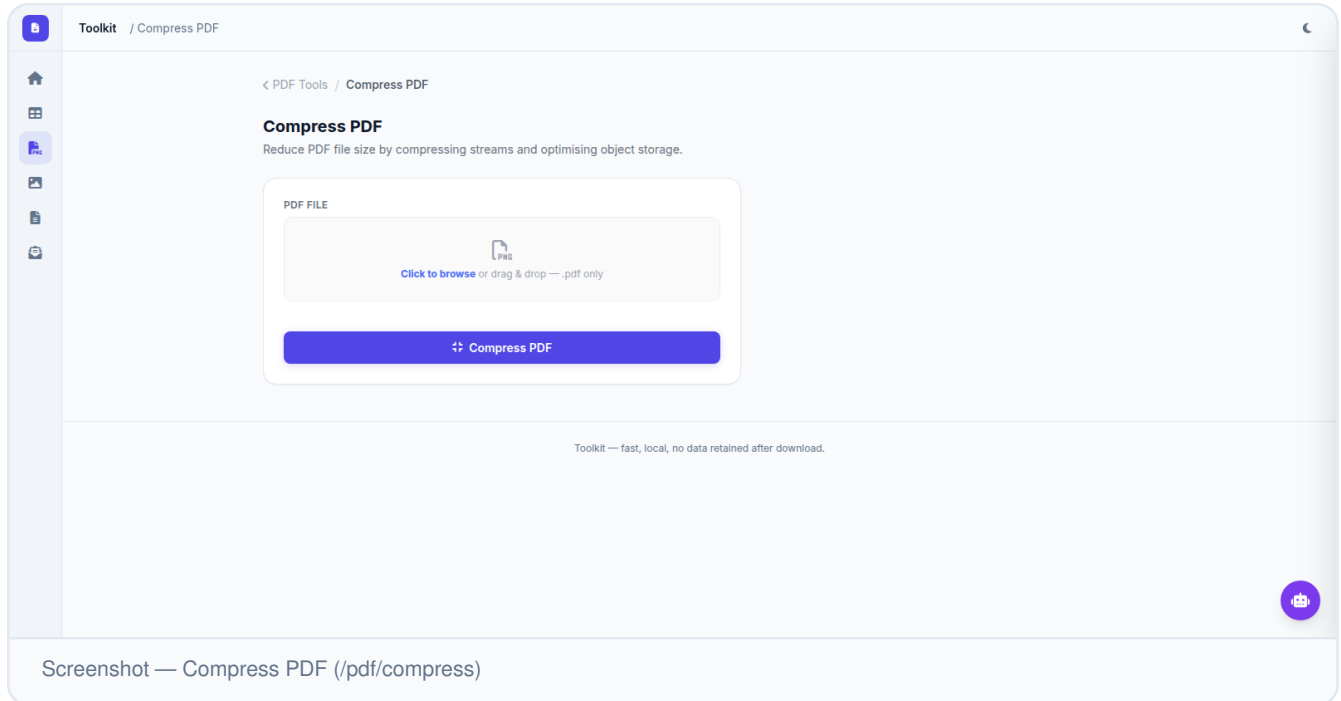
EDITOR'S NOTE

Rebuilding a PDF's page sequence is a non-destructive operation: the source document is read once, then a new PDF is assembled from the selected pages in the specified order. Pages may be referenced multiple times within a single call, enabling duplication without intermediate files. The tool operates on page indices rather than labels, so zero-based or one-based numbering conventions must be confirmed against the implementation before scripting batch workflows. Output file size scales with the number of output pages, not the source, meaning a 200-page document reordered to 10 pages produces a proportionally smaller result. Embedded fonts and annotations are preserved per referenced page but are not de-duplicated across repeated pages.

PDF — Enhance

Compress PDF

Reduces PDF file size by recompressing internal data streams and removing unreferenced objects. No visual quality loss.



KEY FEATURES

- ✓ Recompresses all internal data streams with FlateDecode
- ✓ Packs small objects into object streams for further reduction
- ✓ Removes unreferenced objects and dead cross-reference entries
- ✓ No visual quality loss — purely structural compression
- ✓ Most effective on PDFs with uncompressed or poorly compressed streams
- ✓ UUID output filename
- ✓ Works on all PDF versions 1.0–2.0
- ✓ Returns compressed PDF immediately

HOW IT WORKS

pikepdf saves the PDF with `compress_streams=True` and `object_stream_mode=pikepdf.ObjectStreamMode.generate`. These flags recompress all streams using zlib (FlateDecode) and pack small PDF objects into object

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to compress

OUTPUT

Compressed PDF file. Size reduction varies — text PDFs compress more than image-heavy ones.

LIBRARIES

pikepdf

Flask

USAGE TIPS

1. Text-heavy PDFs typically see 20–60% size reduction.
2. Image-heavy PDFs may see little reduction — use image compression tools first.

streams, significantly reducing size for unoptimised PDFs. Existing well-compressed PDFs see minimal reduction.

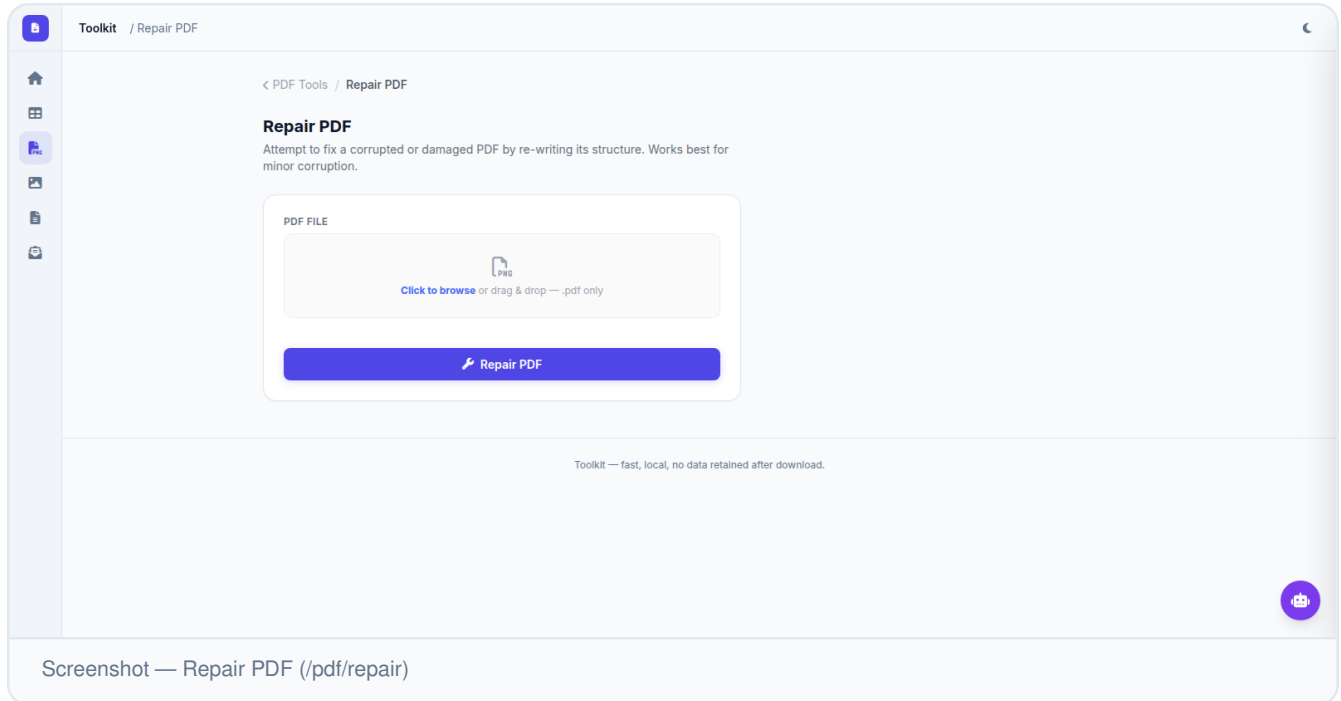
3. Already-optimised PDFs (from Acrobat) may not reduce further.
4. Run OCR before compression if the PDF contains scanned pages.
5. Check the output in a viewer to confirm no content was affected.

EDITOR'S NOTE

Recompressing PDF files without altering their content is a precise operation that demands both correctness and efficiency. This tool applies FlateDecode stream compression and object stream packing via pikepdf, a robust wrapper around the QPDF library, to reduce file size while preserving document fidelity. FlateDecode applies zlib/DEFLATE compression to PDF streams such as page content and embedded data, while object stream packing consolidates indirect objects to reduce overhead. The approach is lossless by design — no image quality is sacrificed. Best practice is to run this tool on unencrypted PDFs; encrypted documents must be decrypted first, as QPDF cannot recompress protected streams without the owner password.

Repair PDF

Attempts to fix corrupted or damaged PDF files by rebuilding the cross-reference table from scratch using pikepdf's fault-tolerant parser.



KEY FEATURES

- ✓ Rebuilds the cross-reference (xref) table from scratch
- ✓ Recovers PDFs that fail to open in standard readers
- ✓ pikepdf scans the entire file for PDF objects when xref is broken
- ✓ Non-fatal parse warnings suppressed during recovery
- ✓ UUID output filename
- ✓ No user parameters required — fully automatic
- ✓ Returns repaired PDF immediately
- ✓ Effective on PDFs with broken xref, wrong offsets, or truncated trailers

HOW IT WORKS

`pikepdf.Pdf.open(path, suppress_warnings=True)` uses pikepdf's fault-tolerant parser which can reconstruct the xref table by scanning the raw file for PDF objects even when the xref section is missing or corrupt. Saving to a

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The damaged or corrupted PDF

OUTPUT

Repaired PDF with rebuilt structure. Severely corrupted content (physical byte loss) cannot be recovered.

LIBRARIES

pikepdf

Flask

USAGE TIPS

1. Try repair first before assuming a PDF is unrecoverable.
2. Repaired PDFs are structurally valid but may still have missing content if bytes were lost.

new path with `pdf.save(out)` writes a fully conformant PDF with a rebuilt xref.

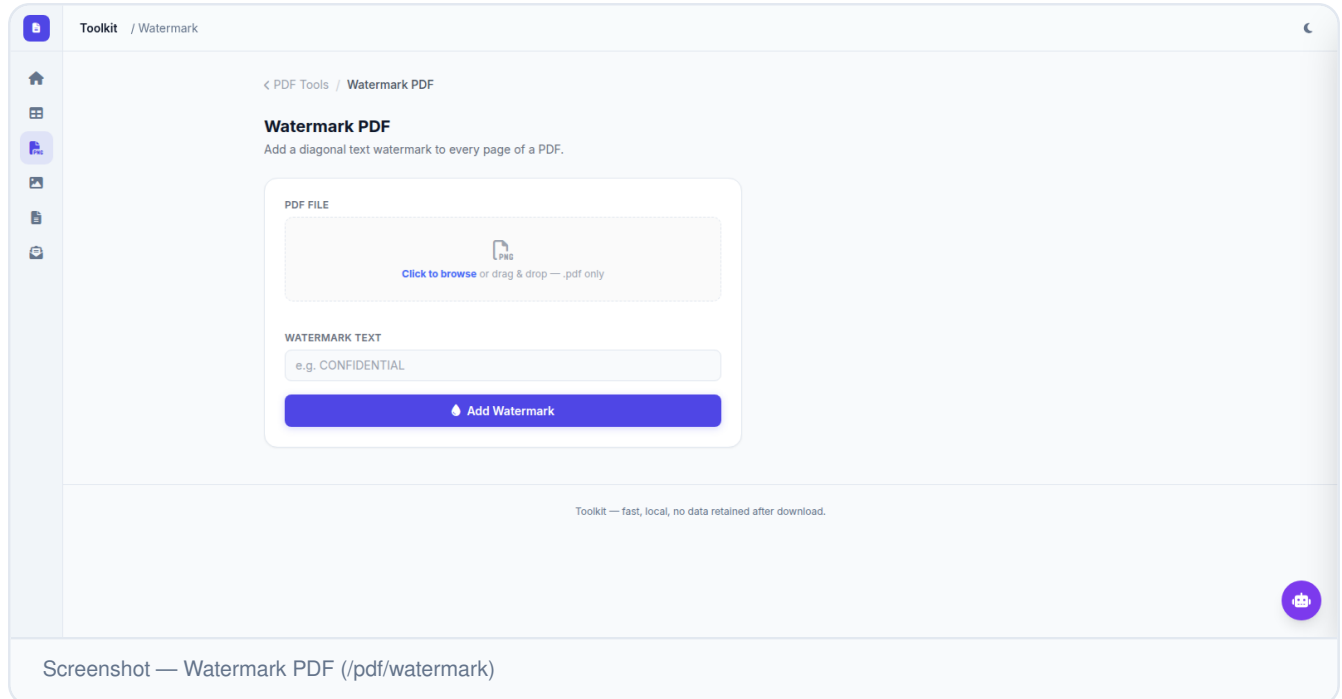
3. Works best on PDFs with logical corruption (bad xref) not physical damage.
4. If repair fails, try opening with a different PDF reader first.
5. Combine with Compress after repair to produce a clean optimised output.

EDITOR'S NOTE

Corrupted PDF files frequently fail to open because their cross-reference table — the internal index that maps object byte offsets — has been truncated, overwritten, or misaligned during an incomplete write or storage failure. This tool rebuilds that structure by passing the file through pikepdf's fault-tolerant parser, which reads object streams directly rather than trusting the existing xref data, then emits a clean, fully linearized replacement. The approach recovers a high proportion of structurally intact PDFs that standard readers reject outright. Practitioners should note that content requiring decryption keys unavailable at parse time will remain inaccessible, and severely fragmented object streams may produce a valid but partially populated output document.

Watermark PDF

Stamps a diagonal semi-transparent text watermark on every page of a PDF. Useful for marking drafts, confidential documents, or proprietary content.



Screenshot — Watermark PDF (/pdf/watermark)

KEY FEATURES

- ✓ Custom watermark text (DRAFT, CONFIDENTIAL, company name, etc.)
- ✓ Diagonal placement at 45° angle across every page
- ✓ Semi-transparent overlay preserves page readability
- ✓ Consistent size and placement regardless of page dimensions
- ✓ UUID output filename
- ✓ Works on PDFs of any page size (A4, Letter, etc.)
- ✓ Returns watermarked PDF immediately
- ✓ Non-destructive — original PDF not modified

HOW IT WORKS

A watermark page is generated using reportlab or fpdf with the text drawn at 45 degrees in grey with partial opacity. pikepdf opens both the source PDF and the watermark page. For each source page, the watermark content stream is merged as an overlay using pikepdf's page merging capability.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to watermark
text	String	Required	Watermark text to display on every page

OUTPUT

PDF with a diagonal text watermark stamped on every page.

LIBRARIES

pikepdf

reportlab

fpdf

Flask

USAGE TIPS

1. Keep watermark text short for best visual impact.

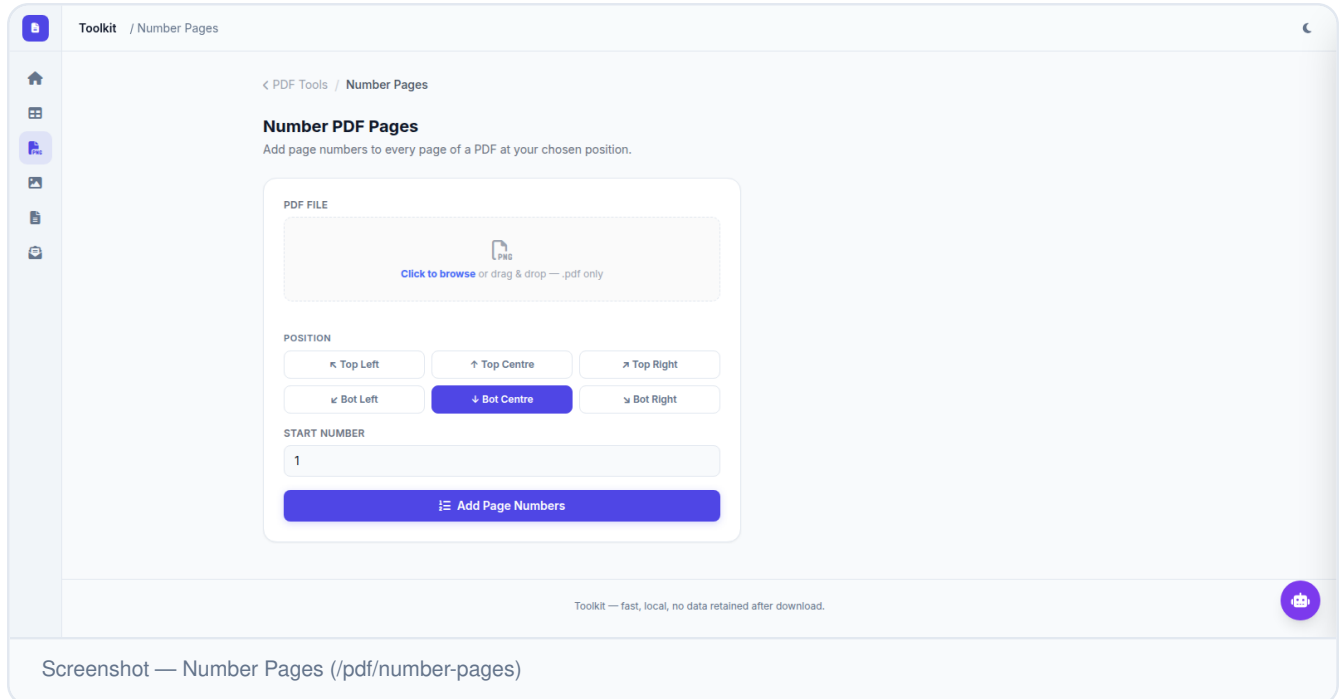
2. "CONFIDENTIAL", "DRAFT", or your company name are common choices.
3. The watermark is visual-only and can be removed with PDF editing tools.
4. Apply watermark after all other edits to avoid double-processing.
5. Use a tool like Acrobat to verify the watermark renders correctly on all pages.

EDITOR'S NOTE

Stamping a diagonal, semi-transparent text watermark across every page of a PDF serves both ownership and workflow purposes — marking drafts as confidential, branding distributed documents, or flagging review copies before final release. This tool renders the watermark at the PDF's native coordinate layer, ensuring it persists regardless of downstream viewers or print drivers. The diagonal orientation maximizes visibility across varied page layouts without obscuring core content, while the semi-transparency preserves readability of the underlying text and graphics. Users should note that the watermark is applied as rendered content, not as a removable PDF annotation layer, so source files should be preserved separately if the unwatermarked version must remain accessible.

Number Pages

Stamps page numbers at a configurable position on every page of a PDF. Supports custom starting number.



Screenshot — Number Pages (/pdf/number-pages)

KEY FEATURES

- ✓ Positions: bottom-center, bottom-right, bottom-left, top-center, top-right, top-left
- ✓ Configurable starting number (default: 1)
- ✓ Small, unobtrusive font that does not obscure content
- ✓ Numbers rendered accurately regardless of page size
- ✓ UUID output filename
- ✓ Consistent positioning across all pages
- ✓ Returns numbered PDF immediately
- ✓ Non-destructive — original PDF not modified

HOW IT WORKS

fpdf or reportlab generates a per-page stamp PDF containing the page number text at the specified position. The position is calculated as a percentage of the page height and width to handle varying page sizes. pikepdf merges each stamp onto the corresponding

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	PDF file	Required	The PDF to number
<code>position</code>	String	Optional	Number position (default: bottom-center)
<code>start</code>	Integer	Optional	Starting page number (default: 1)

OUTPUT

PDF with page numbers stamped on every page.

LIBRARIES

pikepdf

fpdf

Flask

page of the source PDF. The displayed number is `start_number + page_index`.

USAGE TIPS

1. Use a start number > 1 if this PDF follows a front-matter section.
2. Bottom-center is the most conventional position for formal documents.
3. Apply numbering last, after all other edits, to ensure accuracy.
4. Combining with Header & Footer gives a fully formatted professional document.
5. Check the first and last pages to confirm numbering is correct.

EDITOR'S NOTE

Consistent page numbering is essential for any multi-page PDF intended for print, review, or archival. This tool stamps configurable page numbers directly into the PDF content stream at a user-specified position — top or bottom, left, center, or right — across every page in the document. Numbers are rendered as actual PDF text objects rather than annotations, ensuring they survive downstream processing such as flattening or re-distillation. Operators should note that the tool numbers pages sequentially from one by default; documents with existing number annotations or pre-printed folios may produce duplicate numbering, so stripping legacy annotations before processing is strongly recommended for clean results.

Header & Footer

Adds centred header and/or footer text to every page of a PDF. Both fields are optional — supply one or both.

The screenshot shows a web application interface for adding headers and footers to PDFs. It features a sidebar with navigation icons, a main content area with a title 'PDF Header & Footer' and a description 'Add centred header and/or footer text to every page of a PDF.' Below this is a form with a 'PDF FILE' section containing a file upload button and instructions. There are also input fields for 'HEADER TEXT (optional)' and 'FOOTER TEXT (optional)', each with example text. A blue 'Apply Header & Footer' button is at the bottom of the form. The footer of the interface includes the text 'Toolkit — fast, local, no data retained after download.' and a GitHub icon.

Toolkit — fast, local, no data retained after download.

Screenshot — Header & Footer (/pdf/header-footer)

KEY FEATURES

- ✓ Add header text, footer text, or both in one operation
- ✓ Text centred horizontally on every page
- ✓ Consistent vertical positioning regardless of page size
- ✓ Empty fields are simply omitted — no blank stamps
- ✓ UUID output filename
- ✓ Small readable font that does not obscure content
- ✓ Useful for titles, dates, version numbers, or company names
- ✓ Returns modified PDF immediately

HOW IT WORKS

fpdf generates a stamp page with header text positioned near the top and footer text near the bottom. Positions are calculated as percentages of page height to handle varying sizes. pikepdf merges this stamp onto every page of the source PDF. Empty strings for header or footer are checked before rendering.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	PDF file	Required	The PDF to modify
<code>header</code>	String	Optional	Header text (omit to skip)
<code>footer</code>	String	Optional	Footer text (omit to skip)

OUTPUT

PDF with header and/or footer text added to every page.

LIBRARIES

pikepdf

fpdf

Flask

USAGE TIPS

1. Combine with Number Pages for a complete page layout.
2. Use the footer for page numbers and the header for the document title.
3. Keep text short to avoid overflow on narrow pages.
4. Apply after all content edits to prevent double-processing.
5. Test with a single-page PDF first to confirm placement.

EDITOR'S NOTE

Centred text overlays applied via PDF stamp provide a non-destructive, page-consistent branding mechanism without altering original content streams. This tool iterates every page in the source document and composites the supplied header and footer strings at fixed vertical offsets using a transparent stamp layer, preserving the underlying layout intact. The approach suits compliance watermarking, report branding, and draft annotations across multi-page documents. A key design consideration: text is rendered at the overlay layer, so it appears above existing content — operators should verify that headers and footers do not obscure critical document regions, particularly on pages with full-bleed images or top-anchored tables. Font size and margin offsets should be reviewed against the document's established page geometry before batch processing large sets.

Sign PDF

Adds a visible signature block (name, date, reason) to the last page of a PDF. Visual only — not a cryptographic digital signature.

Toolkit / Sign PDF

< PDF Tools / Sign PDF

Sign PDF

Add a visible text signature box to the last page of a PDF.

PDF FILE

Click to browse or drag & drop — .pdf only

SIGNER NAME

e.g. Jane Smith

DATE (optional)

mm/dd/yyyy

REASON (optional)

e.g. Approved

Sign PDF

Screenshot — Sign PDF (/pdf/sign)

KEY FEATURES

- ✓ Adds a professional signature block to the last page
- ✓ Fields: signer name, date, and optional reason
- ✓ Bordered box with name line for a formal appearance
- ✓ Positioned in the bottom-right area of the last page
- ✓ UUID output filename
- ✓ Non-cryptographic — purely visual representation
- ✓ Useful for internal approvals and acknowledgements
- ✓ Returns signed PDF immediately

HOW IT WORKS

reportlab or fpdf generates a signature block: a bordered rectangle containing the signer name, date, reason text, and a horizontal line above the name for a handwritten signature. pikepdf merges this block onto the last page of the source PDF at a fixed offset from the bottom-right corner.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to sign
name	String	Required	Signer's full name
date	String	Optional	Signature date
reason	String	Optional	Reason for signing

OUTPUT

PDF with a visible signature block on the last page.

LIBRARIES

pikepdf

fpdf

Flask

USAGE TIPS

1. This is a visual signature — for legally binding e-signatures use dedicated tools like DocuSign.

2. Add the date to make the signature time-stamped.
3. Use the reason field for approval context (e.g. "Approved for release").
4. Combine with Security to lock the PDF after signing.
5. For multi-page documents, the signature appears only on the last page.

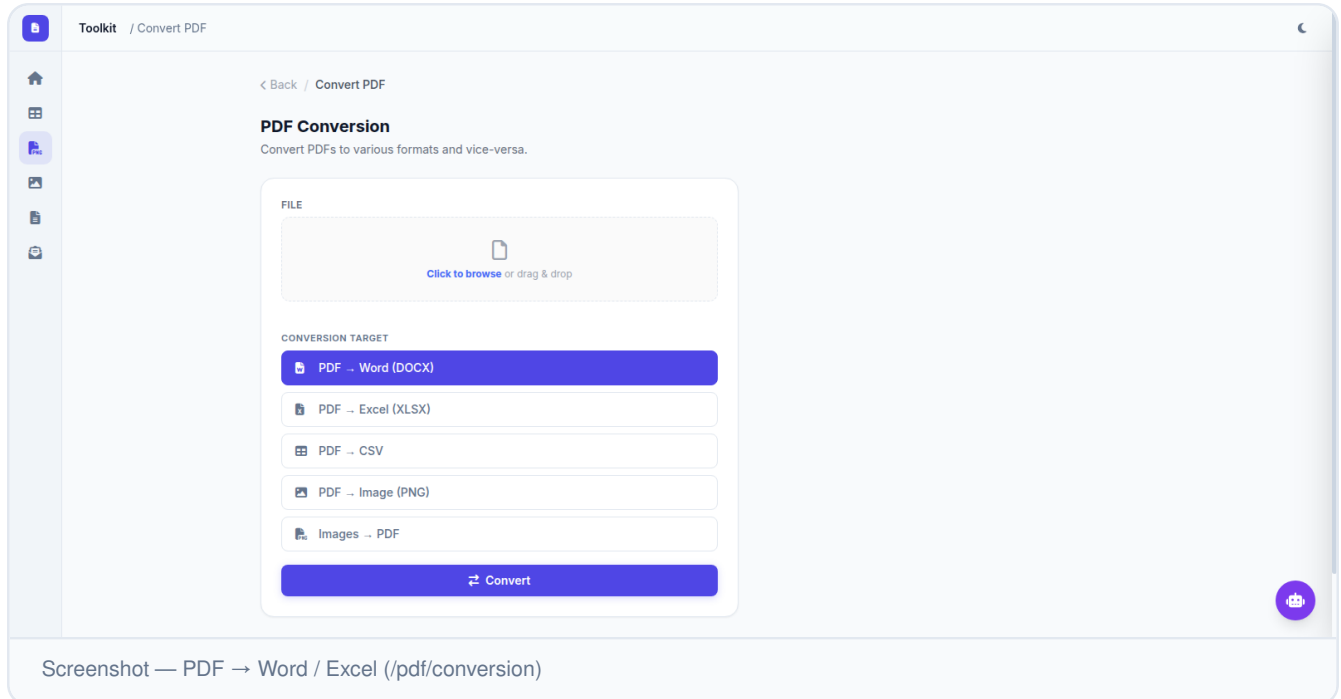
EDITOR'S NOTE

Appending a visible signature block to a PDF's final page, this tool records a signer's name, date, and stated reason directly in the document's content layer. It is designed for workflows where a clear, human-readable attestation matters — audit trails, internal approvals, and document handoffs — without the overhead of certificate infrastructure. Importantly, the signature is purely visual: no cryptographic digest is computed, no certificate chain is embedded, and the output carries no legal equivalence to a qualified electronic signature under frameworks such as eIDAS or E-SIGN. Teams requiring tamper-evident, legally binding signatures should pair this tool with a dedicated signing authority. For documentation and workflow automation purposes, however, the tool provides a reliable, dependency-light solution.

PDF — Convert

PDF → Word / Excel

Converts a PDF to Word (.docx), Excel (.xlsx), CSV, or image formats by extracting text content and rebuilding in the target format.



KEY FEATURES

- ✓ Convert to Word (.docx) — text extracted into paragraphs
- ✓ Convert to Excel (.xlsx) — tabular data written to cells
- ✓ Convert to CSV — tabular content as comma-separated values
- ✓ Convert to images — each page rendered as PNG
- ✓ Text-based PDFs produce best results
- ✓ Scanned PDFs should use OCR first for text output
- ✓ UUID output filename for all formats
- ✓ Returns converted file immediately

HOW IT WORKS

`pdfminer.high_level.extract_text()` extracts all text content. For .docx: `python-docx` creates a Document and writes text blocks as paragraphs. For .xlsx: `openpyxl` creates a workbook and writes text into cells row by row. For .csv: Python `csv` module writes rows.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	PDF file	Required	The PDF to convert
<code>target</code>	docx excel csv image	Required	Output format

OUTPUT

Converted file in the chosen format (docx, xlsx, csv, or PNG images in a ZIP).

LIBRARIES

pdfminer

python-docx

openpyxl

PyMuPDF

Flask

USAGE TIPS

1. Text PDFs (not scanned) convert most accurately.

For images: PyMuPDF renders each page at 150 DPI to PNG.

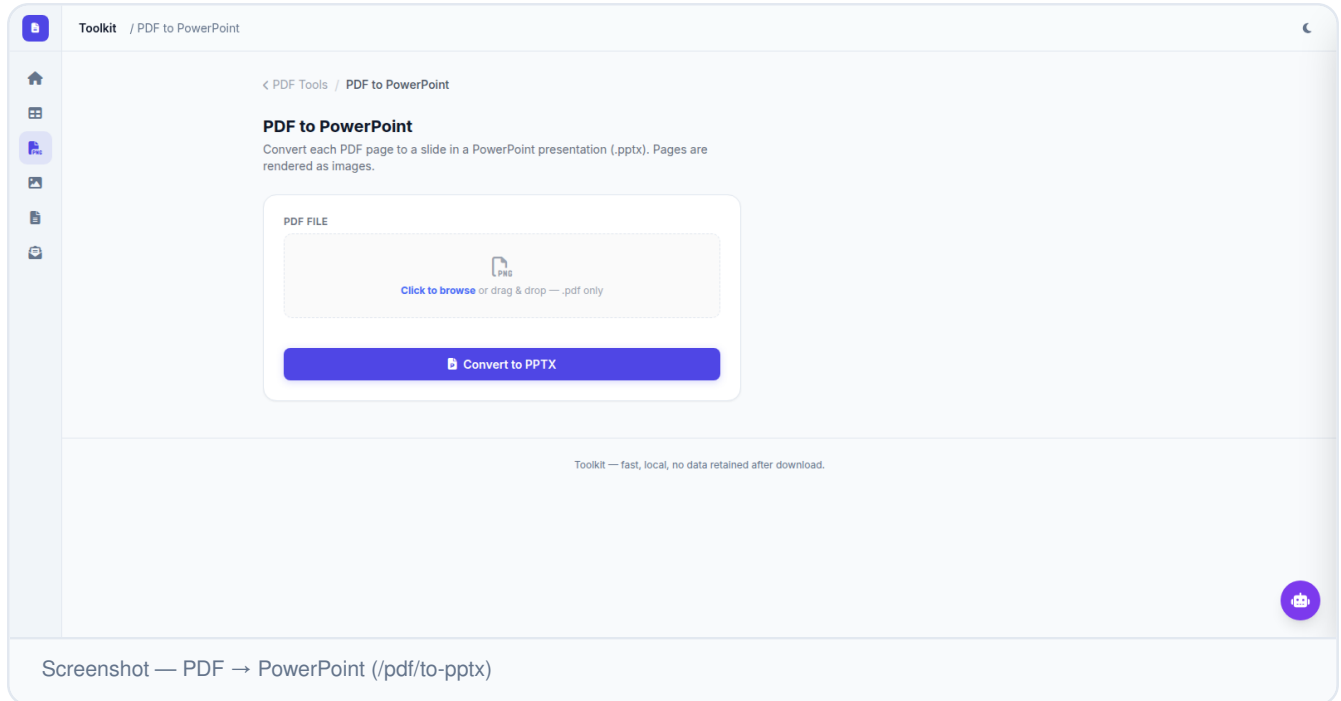
2. Run OCR first on scanned PDFs before converting to Word or Excel.
3. Complex layouts (tables, columns) may not convert perfectly — manual cleanup may be needed.
4. Image conversion is useful for creating slide-ready page thumbnails.
5. CSV output works best for PDFs with simple tabular structure.

EDITOR'S NOTE

Converting PDF documents to editable formats addresses one of the most common document-workflow bottlenecks in enterprise environments. This tool uses pdfminer for text extraction, then routes output through python-docx for DOCX generation, openpyxl for XLSX and CSV construction, or Pillow for rasterized image exports. Because pdfminer operates on the PDF content stream rather than rendering pixels, extraction quality depends heavily on whether the source PDF was digitally authored or produced via scanner; scanned documents require OCR preprocessing before this tool can yield usable text. Tabular content is mapped to spreadsheet rows heuristically, so complex multi-column layouts may require post-conversion cleanup. Best practice is to validate column boundaries on any XLSX output before treating it as authoritative data.

PDF → PowerPoint

Renders each PDF page as a high-quality image and packages them into a PowerPoint (.pptx) file with one slide per page.



KEY FEATURES

- ✓ Each PDF page becomes one PPTX slide
- ✓ Pages rendered at 150 DPI for sharp presentation quality
- ✓ Slide dimensions set to standard widescreen (13.33" × 7.5")
- ✓ Images scaled to fill each slide completely
- ✓ UUID output filename
- ✓ Resulting PPTX is fully editable in PowerPoint or LibreOffice
- ✓ No text extraction — purely visual page-as-image approach
- ✓ Returns PPTX file immediately

HOW IT WORKS

PyMuPDF (fitz) opens the PDF and renders each page as a pixel map at 150 DPI using `page.get_pixmap(matrix=fitz.Matrix(150/72, 150/72))`. Each pixmap is saved as a temporary PNG. `python-pptx` creates a Presentation, adds one slide per page,

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to convert to PowerPoint

OUTPUT

PowerPoint (.pptx) file with one full-page slide per PDF page.

LIBRARIES

PyMuPDF (fitz)

python-pptx

Flask

USAGE TIPS

1. Increase DPI in the source code for higher quality at the cost of file size.
2. Since slides are images, text is not selectable in the PPTX.
3. Use PDF → Word instead if you need editable text.

and inserts each PNG as a full-slide picture using `slide.shapes.add_picture()`.

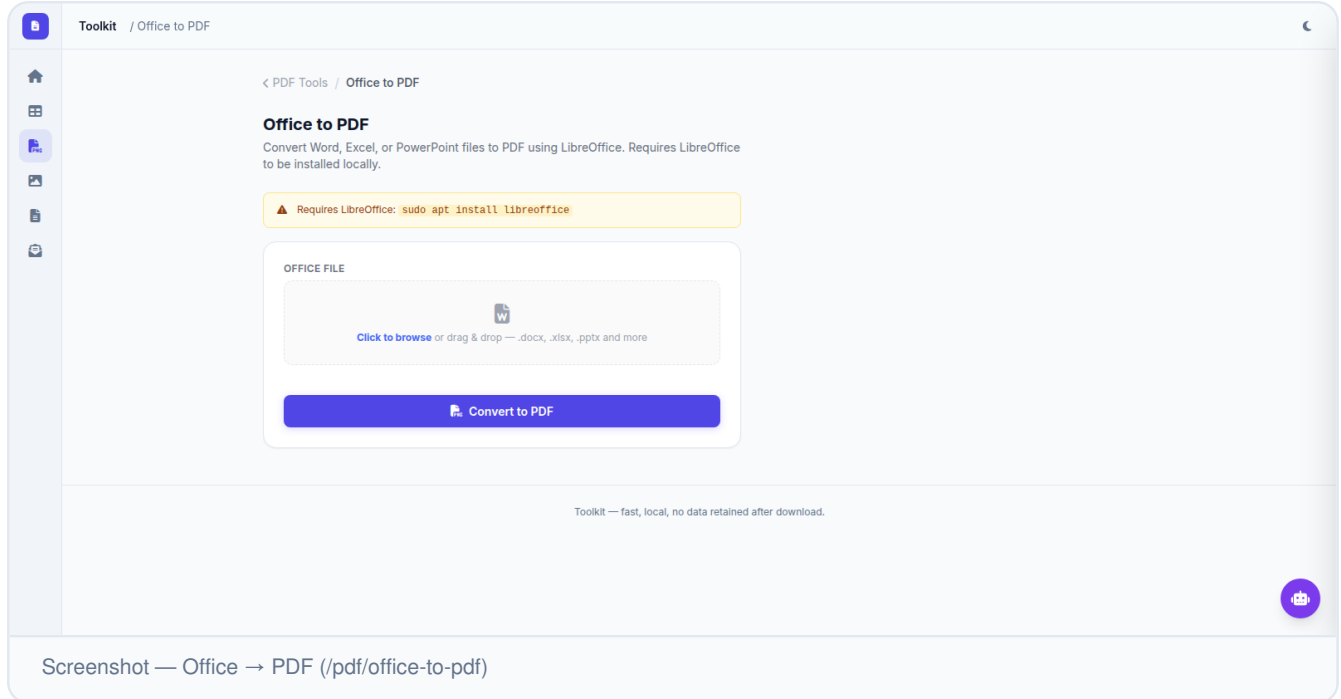
4. Works best for presentations, brochures, and visual documents.
5. The resulting PPTX can be used as a base to add overlays or animations.

EDITOR'S NOTE

Converting PDF documents into editable presentation files, this tool renders each page as a 150 DPI raster image and assembles the results into a PPTX file using `python-pptx`. The 150 DPI setting balances output file size against on-screen clarity; slides are sized to match standard widescreen (16:9) dimensions, so images fill each frame without distortion. Because pages are embedded as images rather than converted to native PowerPoint shapes or text, the output is visually faithful but not directly editable at the content level — downstream text editing requires an OCR post-processing step. For best results, supply vector or high-resolution PDFs; low-quality source documents will produce visibly soft slides.

Office → PDF

Converts Word (.docx/.doc), Excel (.xlsx/.xls), or PowerPoint (.pptx/.ppt) files to PDF using LibreOffice headless rendering.



KEY FEATURES

- ✓ Supports Word (.docx, .doc)
- ✓ Supports Excel (.xlsx, .xls)
- ✓ Supports PowerPoint (.pptx, .ppt)
- ✓ Uses LibreOffice headless for accurate native rendering
- ✓ Preserves fonts, formatting, tables, and layout
- ✓ UUID output filename
- ✓ Returns PDF immediately
- ✓ Requires LibreOffice installed on the server

HOW IT WORKS

werkzeug.secure_filename saves the uploaded Office file to UPLOAD_FOLDER. A subprocess call runs: `libreoffice --headless --convert-to pdf --outdir {dir} {input}`. LibreOffice headless uses its native rendering engine — the same engine used in the desktop app — to produce an accurate PDF.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	Office file	Required	Word (.docx), Excel (.xlsx), or PowerPoint (.pptx) file

OUTPUT

PDF file converted from the Office document with original formatting preserved.

LIBRARIES

subprocess (LibreOffice)

Flask

werkzeug

USAGE TIPS

1. Install LibreOffice on the server: `sudo dnf install libreoffice` or `sudo apt install libreoffice`.

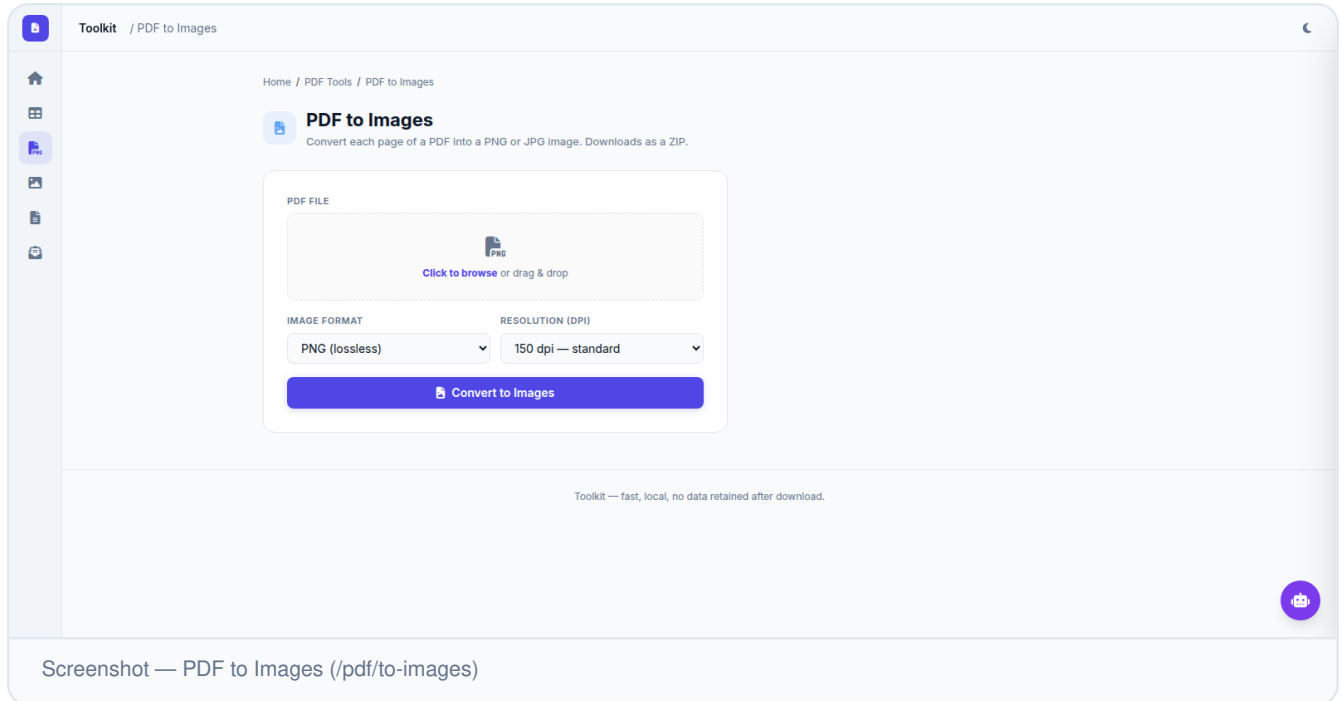
2. Custom fonts in the document must be installed on the server for accurate rendering.
3. Excel multi-sheet workbooks are converted to multi-page PDFs.
4. PowerPoint animations are not preserved in the PDF output.
5. Test with a sample file first to verify font rendering on your server.

EDITOR'S NOTE

Converting Word, Excel, and PowerPoint documents to PDF is handled by invoking LibreOffice in headless subprocess mode, which eliminates any dependency on proprietary document renderers or cloud services. This approach faithfully preserves fonts, tables, charts, and page layout by delegating rendering to LibreOffice's mature document engine rather than reimplementing format parsing. Because each conversion spawns a short-lived subprocess, the tool is stateless and safe to run concurrently across requests, though operators should cap parallelism to avoid exhausting system memory on large files. LibreOffice must be installed on the host or container image; the recommended practice is to pin the package version in the Dockerfile to prevent silent rendering regressions after upstream updates.

PDF to Images

Renders every page of a PDF as a PNG or JPG image and returns all images in a single ZIP archive. Supports three DPI levels for screen, standard, or print quality.



Screenshot — PDF to Images (/pdf/to-images)

KEY FEATURES

- ✓ Converts every page to PNG (lossless) or JPG
- ✓ Three DPI options: 72 (screen), 150 (standard), 300 (print quality)
- ✓ All page images returned in a single ZIP archive
- ✓ Pages named sequentially: page_001.png, page_002.png etc.
- ✓ UUID-named temporary directory prevents concurrent-run conflicts
- ✓ Powered by PyMuPDF — accurate and fast rendering
- ✓ No watermarks or visual modifications to page content
- ✓ Returns ZIP immediately after conversion

HOW IT WORKS

PyMuPDF (fitz) opens the PDF. A transformation matrix is computed as `fitz.Matrix(dpi/72, dpi/72)`. For each page, `page.get_pixmap(matrix=mat, alpha=False)` renders it to a pixel map. Each pixmap is saved to a

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to convert
fmt	png jpg	Optional	Image format (default: png)
dpi	72 150 300	Optional	Rendering resolution (default: 150)

OUTPUT

ZIP archive of page images named page_001.{fmt} through page_N.{fmt}.

LIBRARIES

PyMuPDF (fitz)

zipfile

Flask

UUID-named temporary directory. `zipfile.ZipFile` compresses all images into a ZIP returned as a download.

USAGE TIPS

1. Use 300 DPI for print-quality output or for feeding into OCR tools.
2. PNG preserves exact pixels with no compression artifacts.
3. JPG is significantly smaller but introduces compression artifacts.
4. Large PDFs at 300 DPI produce large files — allow extra time.
5. Use the resulting images in presentations, thumbnails, or image processing pipelines.

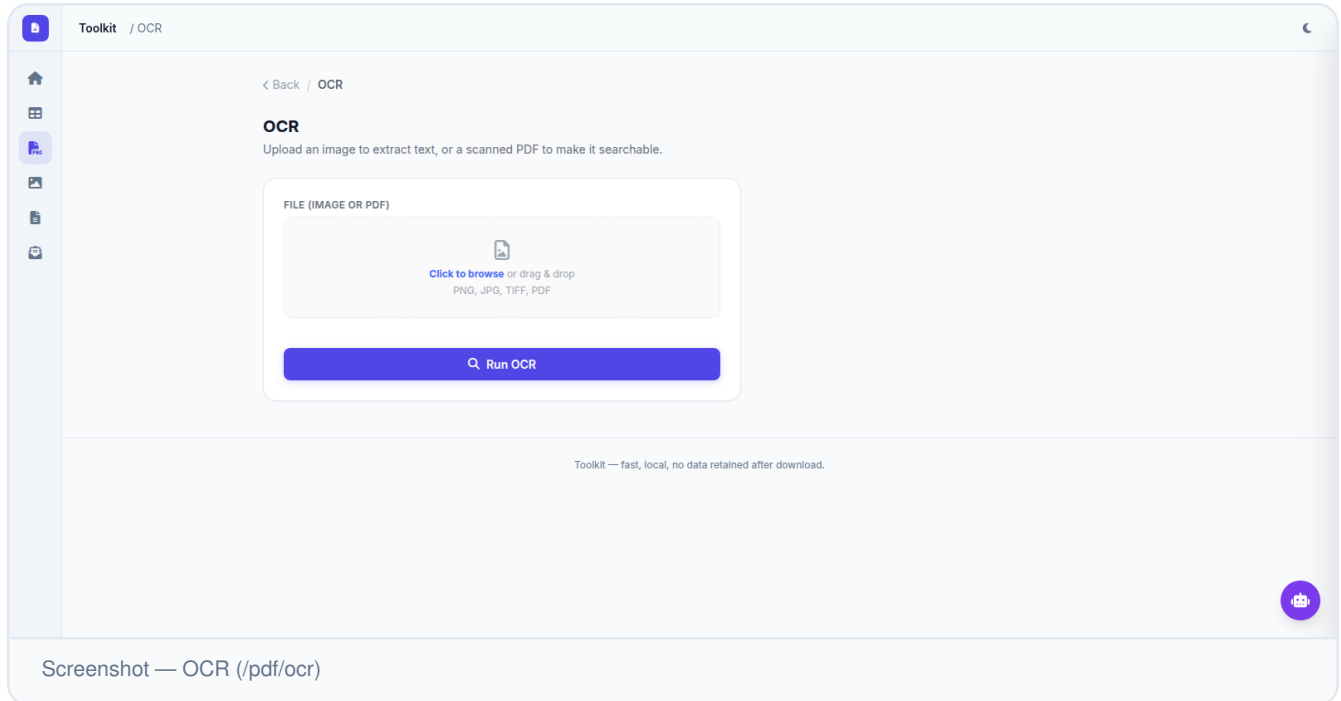
EDITOR'S NOTE

Converting PDF documents to raster images is a common requirement for thumbnail generation, document previews, OCR pipelines, and archival workflows. This tool renders each PDF page to PNG or JPG format using PyMuPDF (`fitz`), a high-performance binding to the MuPDF rendering engine known for accurate font handling and color fidelity. The configurable DPI parameter directly controls output resolution: 72 DPI produces screen-quality previews, while 150–300 DPI suits print or OCR workloads. All rendered pages are packaged into a single ZIP archive for convenient download. Callers should note that high-DPI rendering of large or complex PDFs is memory-intensive; multi-hundred-page documents at 300 DPI may require significant server resources and extended processing time.

PDF —
Other

OCR

Extracts text from scanned PDFs and images using the Tesseract OCR engine. Supports multi-page PDFs with per-page text extraction.



KEY FEATURES

- ✓ Extracts text from scanned PDF files page by page
- ✓ Extracts text from PNG, JPG, and WEBP images
- ✓ Powered by Tesseract OCR via pytesseract
- ✓ Multi-page PDFs: each page rendered at 200 DPI then OCR'd
- ✓ All page results concatenated with page separators
- ✓ Output displayed in browser for immediate copy-paste
- ✓ No output file created — result served as plain text
- ✓ Accuracy depends heavily on input image quality

HOW IT WORKS

For images: `PIL.Image.open()` loads the file, `pytesseract.image_to_string()` runs OCR. For PDFs: `PyMuPDF (fitz)` renders each page as a 200 DPI pixmap, converts to a PIL Image, and feeds to `pytesseract`. All page results are joined with "--- Page N ---" separators and returned as plain text in the browser.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF or image	Required	Scanned PDF, PNG, JPG, or WEBP file

OUTPUT

Extracted text displayed in the browser. Copy and paste for further use.

LIBRARIES

pytesseract

Tesseract (system)

PyMuPDF

Pillow

Flask

USAGE TIPS

1. Higher resolution scans (300 DPI+) produce more accurate OCR results.
2. Clean, high-contrast documents give the best text extraction quality.

3. Tesseract supports multiple languages — set lang parameter in the code for non-English text.
4. OCR results are approximate — always proofread extracted text.
5. Convert scanned PDFs to searchable PDFs using a dedicated tool like ocrmypdf after extraction.

EDITOR'S NOTE

Optical character recognition in this toolkit is handled by a dedicated OCR module that wraps Tesseract via the pytesseract binding, enabling text extraction from scanned PDFs and raster images without requiring a full document processing pipeline. The tool accepts both image files and PDF inputs, converting PDF pages to images internally before passing them to the Tesseract engine. This design keeps the interface uniform regardless of source format. Because Tesseract's accuracy degrades significantly with low-resolution scans, skewed text, or unusual fonts, callers should preprocess images to at least 300 DPI and apply deskewing where possible. Output is plain UTF-8 text; no layout or bounding-box metadata is preserved by default.

Security

Password-protects a PDF with AES-256 encryption or unlocks a password-protected PDF given its password.

Toolkit / Security

< Back / PDF Security

PDF Security

Password-protect a PDF or remove its password.

ACTION

☒ Protect ☐ Unlock

PDF FILE

Click to browse or drag & drop — .pdf only

PASSWORD

Enter password

Submit

Toolkit — fast, local, no data retained after download.

Screenshot — Security (/pdf/security)

KEY FEATURES

- ✓ Protect with AES-256 encryption (PDF 2.0, R=6)
- ✓ Both owner and user passwords set for simplicity
- ✓ Unlock a protected PDF given its correct password
- ✓ pikepdf uses standards-compliant encryption
- ✓ UUID output filename
- ✓ Returns protected or unlocked PDF immediately
- ✓ Works with PDF versions 1.4 and above
- ✓ Encrypted PDFs require the password to open in any viewer

HOW IT WORKS

For protect: `pikepdf.Encryption(owner=password, user=password, R=6)` configures AES-256 (R=6 = PDF 2.0 spec). `pdf.save(out, encryption=enc_spec)` writes the encrypted file. For unlock: `pikepdf.Pdf.open(path, password=password)` opens the file using the provided password; `pdf.save(new_path)` saves without encryption.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
file	PDF file	Required	The PDF to protect or unlock
action	protect unlock	Required	Operation to perform
password	String	Required	Password to set (protect) or to use (unlock)

OUTPUT

Password-protected or unlocked PDF file.

LIBRARIES

pikepdf

Flask

USAGE TIPS

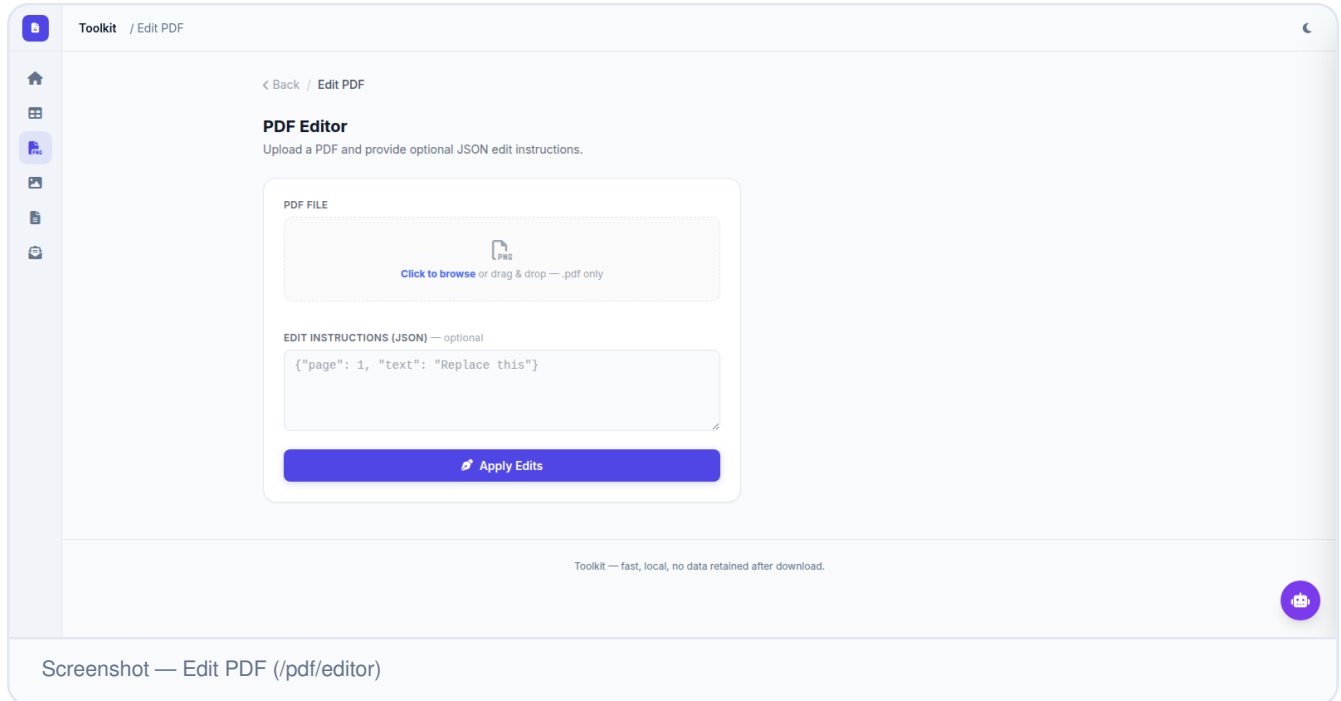
1. Use a strong password — AES-256 is strong but a weak password is easy to guess.
2. Store your password securely — there is no recovery mechanism.
3. Protecting a PDF prevents editing and copying in most PDF readers.
4. Owner password can be set differently from user password in the source code for finer control.
5. Unlock requires the correct password — wrong passwords are rejected with an error.

EDITOR'S NOTE

Protecting and unlocking PDF files is handled through the Security tool, which leverages pikepdf's AES-256 encryption API to apply or remove password protection at the document level. AES-256 is the current industry standard for symmetric encryption, making it suitable for sensitive documents requiring compliance with data-handling policies. When encrypting, the tool distinguishes between owner and user passwords, allowing fine-grained control over printing, copying, and editing permissions. Decryption requires the correct owner or user credential and will fail loudly rather than silently on incorrect input. Practitioners should note that pikepdf does not support legacy RC4-encrypted PDFs uniformly across all versions; upgrading encryption on older documents before applying new restrictions is the recommended best practice.

Edit PDF

Applies AI-guided text edits to a PDF using a local Ollama language model. Describe the edit in plain English and the AI applies it.



KEY FEATURES

- ✓ Describe edits in plain English — no technical knowledge needed
- ✓ AI interprets intent and applies changes via Ollama
- ✓ Extracts existing PDF text as context for the LLM
- ✓ Rebuilds PDF with the AI-modified content
- ✓ Powered by local Ollama — fully private, no cloud API
- ✓ Supports text replacement, rewording, and summarisation
- ✓ Returns modified PDF immediately
- ✓ Requires Ollama running at localhost:11434

HOW IT WORKS

`pdfminer.high_level.extract_text()` extracts the PDF text. The extracted content plus the user instruction are packaged into a structured prompt sent to the Ollama REST API (POST `/api/chat`). The LLM returns modified text which `fpdf` or `reportlab` uses to rebuild the PDF. The entire pipeline runs locally.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	PDF file	Required	The PDF to edit
<code>instruction</code>	String	Required	Plain-language edit instruction

OUTPUT

Modified PDF with AI-applied changes. Review carefully before use.

LIBRARIES

`pdfminer``Ollama API``fpdf``Flask`

USAGE TIPS

1. Be specific in your instruction: "Replace all occurrences of 'Company A' with 'Company B'".

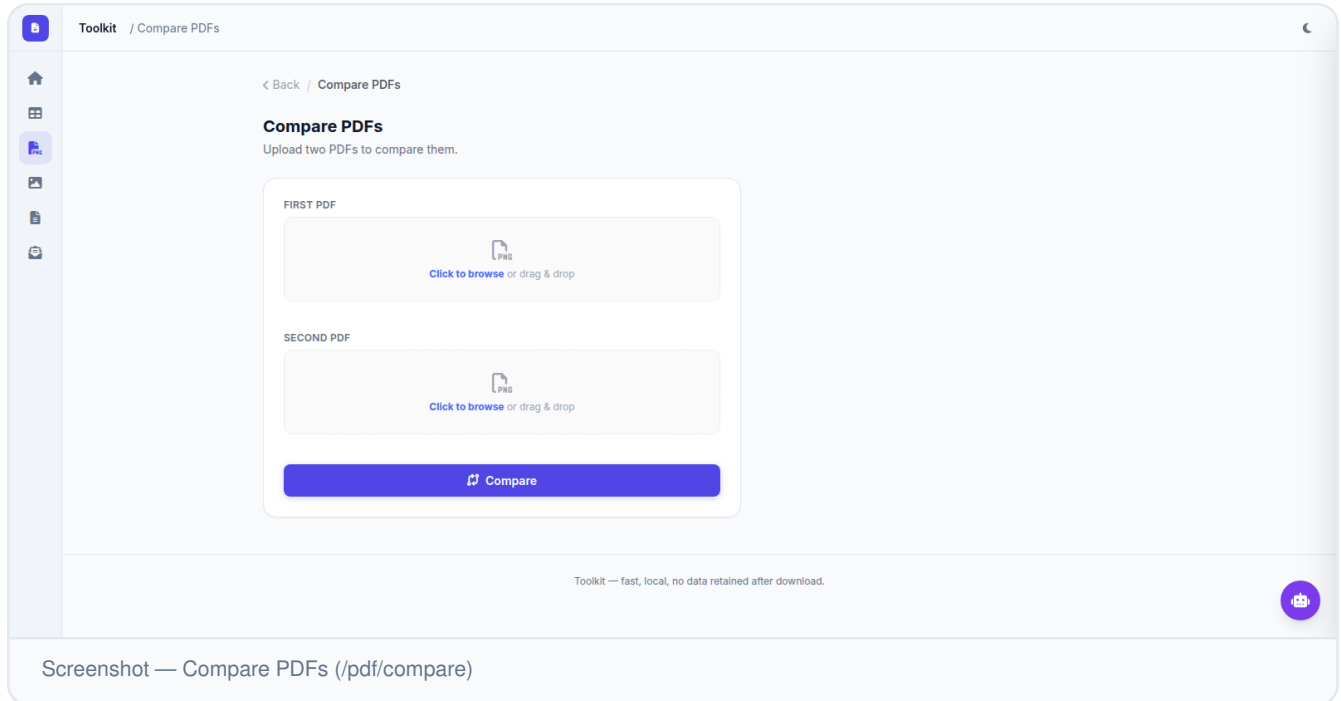
2. Complex layout PDFs (multi-column, tables) rebuild as plain text.
3. Always review AI edits — the model may make unexpected changes.
4. Ensure Ollama is running before using this tool.
5. Works best on simple, text-heavy PDFs.

EDITOR'S NOTE

Powered by a locally running Ollama LLM, this tool accepts plain-English instructions and applies targeted text edits to an existing PDF without requiring cloud services or API keys. It re-renders only the affected content, preserving the document's structure, fonts, and layout where possible. The key design decision is offline execution: all inference runs on the user's machine via the Ollama endpoint, which satisfies privacy requirements for sensitive documents. Practitioners should note that edit fidelity depends on the underlying model's instruction-following capability; for complex structural changes or multi-column layouts, results should be reviewed and validated before the modified PDF is used in production workflows.

Compare PDFs

Compares two PDF files and reports differences in file size and text content. Useful for reviewing document revisions.



Screenshot — Compare PDFs (/pdf/compare)

KEY FEATURES

- ✓ Compares two PDF files side by side
- ✓ Reports file size difference between the two
- ✓ Extracts and diffs text content from both files
- ✓ Identifies added and removed text blocks
- ✓ Works on text-based PDFs (not scanned)
- ✓ Results displayed in the browser
- ✓ No output file — result served as HTML report
- ✓ Useful for reviewing document revisions

HOW IT WORKS

Both files are saved to `UPLOAD_FOLDER`. `pdfminer.high_level.extract_text()` extracts text from each. A comparison is performed on file size and text content. Results are formatted as a readable summary returned in the HTML response showing size difference and text changes.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file1</code>	PDF file	Required	First PDF (reference version)
<code>file2</code>	PDF file	Required	Second PDF (new version to compare)

OUTPUT

Diff report displayed in the browser showing size and text content differences.

LIBRARIES

pdfminer

pikepdf

Flask

USAGE TIPS

1. Text-based PDFs give the most meaningful diffs.
2. Scanned PDFs show only file size differences — use OCR first.

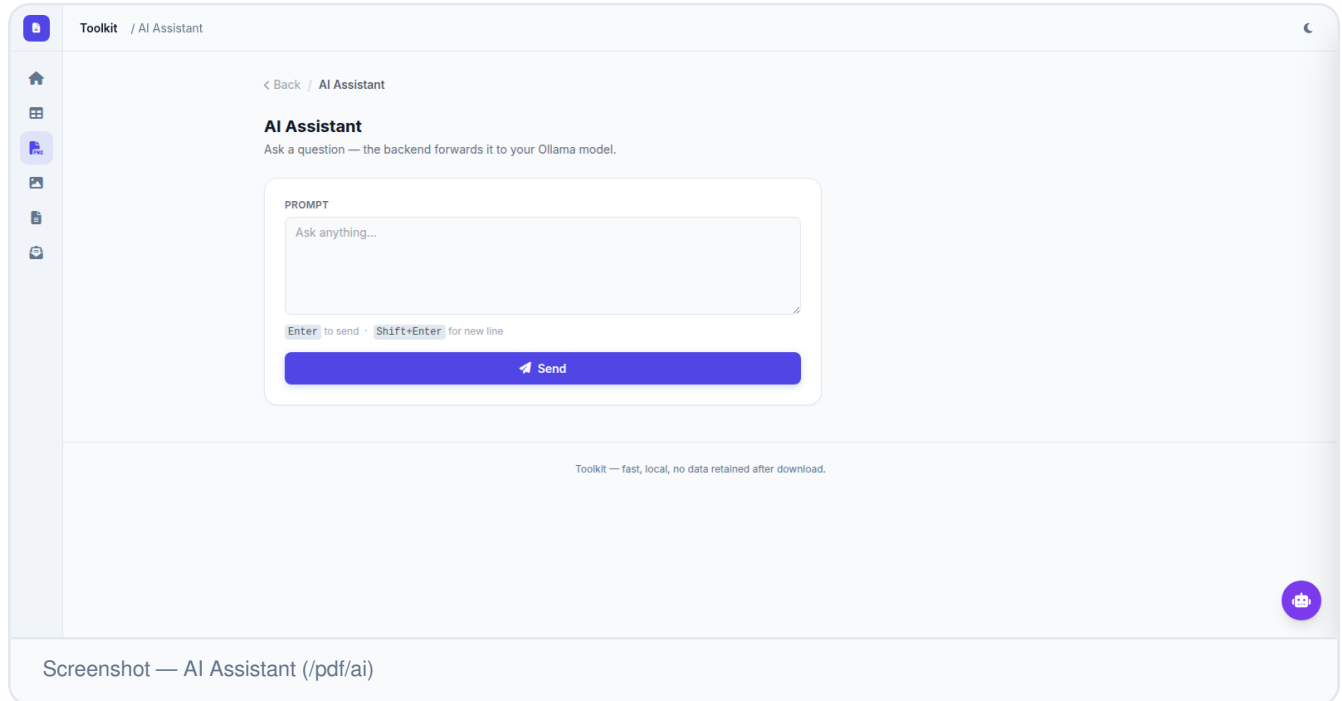
3. Minor formatting changes may not appear in the text diff.
4. Compare before and after editing to verify changes were applied correctly.
5. Use alongside the Edit PDF tool to verify AI edits.

EDITOR'S NOTE

Diffing two PDF documents at both the file-size and text-content levels, this tool gives developers and analysts a fast, deterministic way to detect regressions, verify re-exports, and audit pipeline outputs without manual inspection. Text extraction is handled by pdfminer, which parses the PDF content stream directly rather than relying on OCR, making results accurate for machine-generated documents but unreliable for scanned images or heavily embedded fonts. The file-size comparison surfaces encoding or compression differences that text diffing alone would miss. Users should note that cosmetic changes — page layout shifts, font substitutions, metadata updates — may alter file size without affecting extracted text, so both metrics should be evaluated together for a complete picture.

AI Assistant

A local AI chat interface powered by Ollama. Attach a PDF, CSV, or text file as context and ask questions about it.



KEY FEATURES

- ✓ Chat interface with a local Ollama language model
- ✓ Attach PDF, CSV, or text files as conversation context
- ✓ File content extracted and prepended to the prompt
- ✓ Supports multi-turn conversation with history
- ✓ Fully local — no data sent to any cloud API
- ✓ Model configurable — uses whichever model is running in Ollama
- ✓ Rate limit handling with user-friendly error message
- ✓ Markdown formatting in responses

HOW IT WORKS

User message plus optional file context is sent to the Ollama REST API: POST localhost:11434/api/chat with {model, messages}. For PDFs, pdfminer extracts text; for CSVs, pandas reads and formats a data preview. The file content is injected as a system message. Conversation history is maintained for multi-turn interaction.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
question	String	Required	Your question or instruction
file	PDF/ CSV/ text	Optional	File to use as context for the question

OUTPUT

AI response displayed in the chat interface. Supports markdown formatting.

LIBRARIES

Ollama REST API

requests

pdfminer

pandas

Flask

USAGE TIPS

1. Start Ollama before using: ollama serve.

2. Attach a CSV and ask "summarise this data" for instant insights.
3. Ask follow-up questions — the conversation history is maintained.
4. Be specific: "What are the key dates in this document?" gives better results than "summarise".
5. Large files may exceed the model's context window — summarise first if needed.

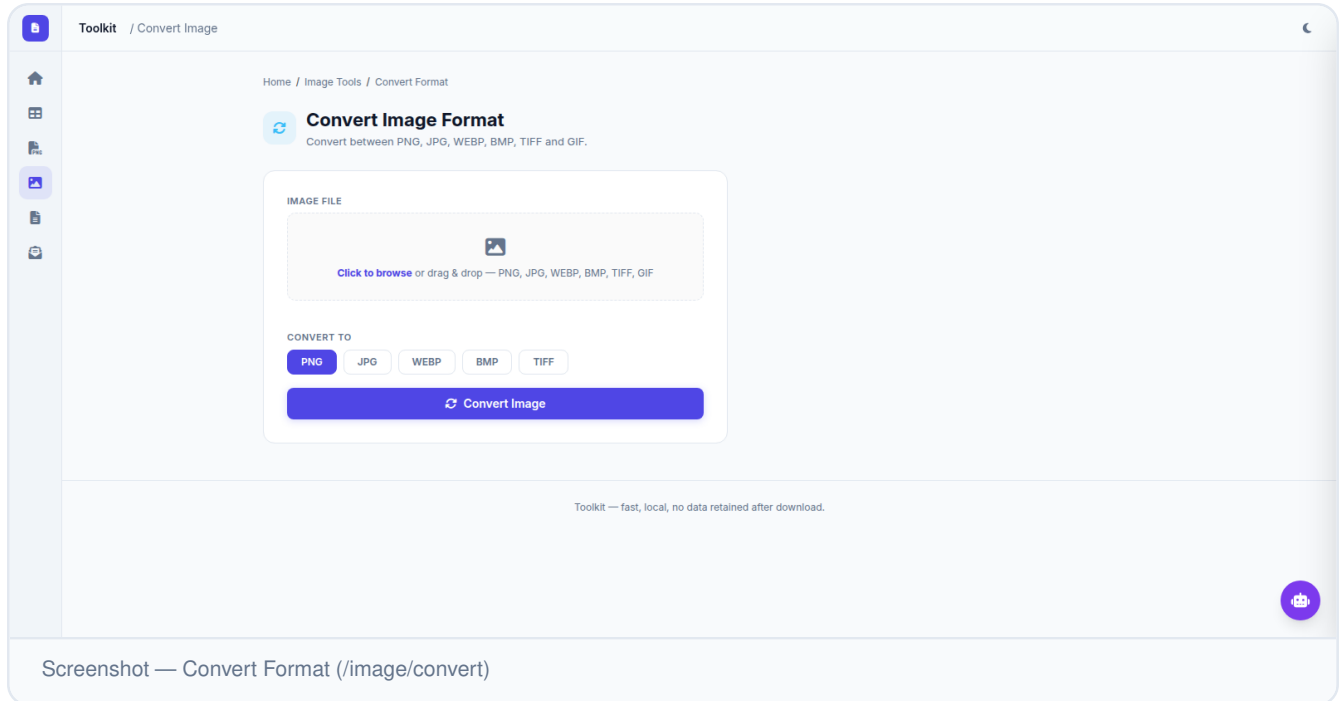
EDITOR'S NOTE

Powered by a locally-hosted Ollama instance, this tool provides a conversational interface for querying structured and unstructured documents — PDFs, CSVs, and plain text — without transmitting data to external services. File context is injected directly into the prompt, making the tool well-suited for sensitive or proprietary content that must remain on-premises. The default model is phi-3, selected for its balance of response quality and local resource consumption. Practitioners should note that context window limits constrain how much of a large document can be processed in a single query; for lengthy files, chunking input manually or summarizing sections beforehand yields more reliable answers. No persistent memory exists between sessions.

Image

Convert Format

Converts an image between PNG, JPG, WEBP, BMP, and TIFF formats. Handles mode conversion (RGBA→RGB) automatically for JPEG output.



Screenshot — Convert Format (/image/convert)

KEY FEATURES

- ✓ Converts between PNG, JPG, WEBP, BMP, and TIFF
- ✓ RGBA to RGB conversion applied automatically for JPEG
- ✓ LANCZOS resampling preserves image quality
- ✓ UUID output filename prevents collisions
- ✓ Before/after comparison viewer
- ✓ Returns converted image immediately
- ✓ Supports drag-and-drop upload
- ✓ Handles all Pillow-supported colour modes

HOW IT WORKS

Pillow `Image.open()` loads the image. If the target format is JPEG and the image has an alpha channel (RGBA or LA mode), it is converted to RGB with `Image.convert("RGB")` to avoid JPEG compatibility errors. `Image.save(out_path, format=target)` writes the output. UUID prefix on output filename. Served via `send_file` with correct MIME type.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	Image file	Required	Any supported image format
<code>format</code>	String	Required	Target format: png, jpg, webp, bmp, or tiff

OUTPUT

Converted image in the chosen format with UUID-prefixed filename.

LIBRARIES

Pillow (PIL)

Flask

USAGE TIPS

1. PNG is lossless — use for screenshots and graphics with sharp edges.

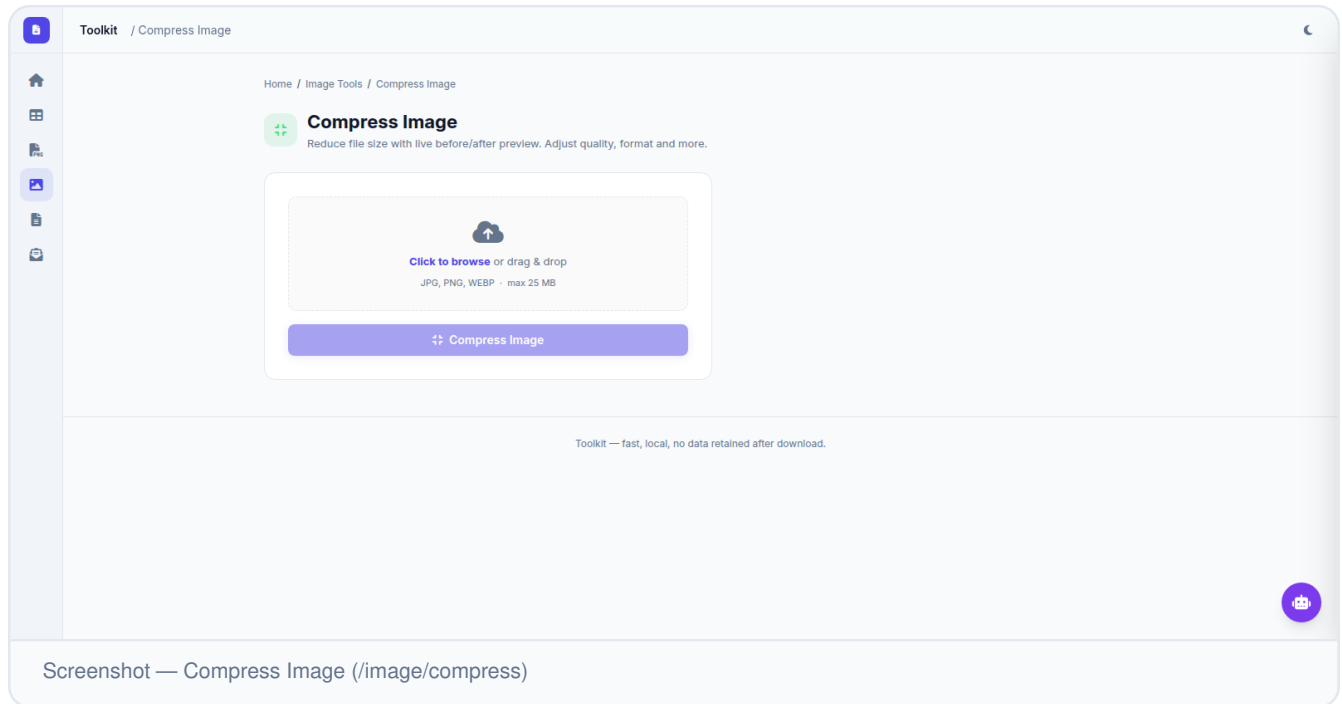
2. JPG is lossy but produces much smaller files for photographs.
3. WEBP gives the best size/quality ratio for web use.
4. Converting PNG with transparency to JPG fills the alpha channel with white.
5. BMP and TIFF are uncompressed — use only when required by another application.

EDITOR'S NOTE

Format conversion in the Toolkit handles cross-format image translation across PNG, JPG, WEBP, BMP, and TIFF using Pillow's native codec pipeline. Auto mode conversion is applied before encoding, ensuring that images with transparency (RGBA or palette modes) are safely normalized when writing to formats that lack alpha-channel support, such as JPG. This prevents silent corruption or encoding errors that commonly occur when naive pipelines skip mode checks. WEBP is generally recommended as the output target when file size matters, as it offers superior compression to both PNG and JPG at comparable quality. Lossless fidelity is preserved for PNG and TIFF outputs, making those formats appropriate for archival or pre-processing pipeline stages.

Compress Image

Reduces image file size with a two-step interactive flow. Upload once, then fine-tune quality, format, resize, brightness, contrast, sharpness, and blur — all with live before/after comparison. Supports JPG, PNG, and WEBP up to 25 MB.



Screenshot — Compress Image (/image/compress)

KEY FEATURES

- ✓ Step 1: upload image (max 25 MB — JPG, PNG, WEBP) with XHR progress bar
- ✓ Step 2: live side-by-side Before / After viewer with overlay spinner during re-processing
- ✓ Quality slider 1–100 (default 72); settings auto-apply 800 ms after any change
- ✓ Output format picker: JPG, PNG, WEBP — PNG warning shown as PNG is lossless and larger
- ✓ Advanced — Resize: None / By % (10–200%) / Custom px with aspect-ratio lock button
- ✓ Advanced — Brightness slider 0.1–3.0 (PIL ImageEnhance.Brightness)

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	Image (JPG/PNG/WEBP)	Required	Image file — max 25 MB
<code>quality</code>	Integer 1–100	Optional	Compression quality (default: 72)
<code>format</code>	jpg png webp	Optional	Output format (default: matches input)
<code>resize_mode</code>	none percentage custom	Optional	Resize strategy (default: none)
<code>resize_percent</code>	Integer 10–200	Conditional	Scale % when <code>resize_mode=percentage</code>
<code>resize_width</code>	Integer px	Conditional	Output width when <code>resize_mode=custom</code>
<code>resize_height</code>	Integer px	Conditional	Output height when <code>resize_mode=custom</code>
<code>lock_aspect</code>	Boolean	Optional	

- ✓ Advanced — Contrast slider 0.1–3.0 (PIL ImageEnhance.Contrast)
- ✓ Advanced — Sharpness slider 0.0–3.0 (PIL ImageEnhance.Sharpness)
- ✓ Advanced — Blur slider 0–20 radius (PIL GaussianBlur)
- ✓ Manual Apply button + Reset button restores all controls to defaults
- ✓ Stats bar shows original size/dims → result size/dims + % reduction badge
- ✓ UUID file IDs link original and compressed pair across both steps

HOW IT WORKS

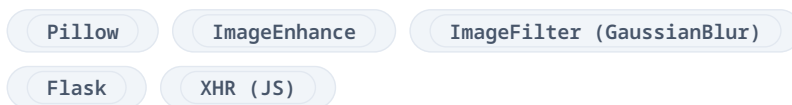
Step 1 (POST /image/compress-upload): file uploaded via XHR with progress tracking, saved as {uuid}_orig.ext, immediately processed at quality=60 via _process_image(), returns JSON with file_id, sizes, dimensions, and serve URLs. Step 2 (POST /image/compress-process): receives JSON settings {file_id, quality, format, resize_mode, resize_percent, resize_width, resize_height, lock_aspect, brightness, contrast, sharpness, blur}, re-processes the stored original using _process_image() which applies PIL ImageEnhance for brightness/contrast/sharpness, GaussianBlur for blur, and LANCZOS resize — then saves the compressed output as {uuid}_comp.ext. Returns updated JSON stats. The JS schedules auto-apply 800 ms after any slider change using setTimeout.

FIELD	TYPE	REQUIRED	DESCRIPTION
			Lock aspect ratio for custom resize (default: true)
brightness	Float 0.1–3.0	Optional	Brightness multiplier (default: 1.0)
contrast	Float 0.1–3.0	Optional	Contrast multiplier (default: 1.0)
sharpness	Float 0.0–3.0	Optional	Sharpness multiplier (default: 1.0)
blur	Float 0–20	Optional	Gaussian blur radius (default: 0)

OUTPUT

Compressed image served at /image/dl-comp/{uuid}_comp.ext. JSON response includes compressed_size_str, original_size_str, reduction_percent, output_width, output_height, compressed_url, and download_url.

LIBRARIES



USAGE TIPS

1. Quality 65–80 gives the best size/quality balance for photographs.
2. The original file is never overwritten — re-adjust settings freely without re-uploading.
3. WEBP at quality 72 is typically 30–50% smaller than an equivalent JPG.
4. PNG is lossless — the tool warns you if you select it, as file size will be larger than JPG.
5. Use resize + quality together for maximum file size reduction (e.g. scale to 50% + quality 70).
6. Sharpness above 1.5 compensates for softness introduced by heavy compression.
7. Blur at radius 1–2 can smooth compression artefacts on very low quality settings.
8. Click Reset to restore all sliders to defaults without re-uploading the image.

Resize Image

Scales an image to specified pixel dimensions. Optionally locks the aspect ratio to prevent distortion.

KEY FEATURES

- ✓ Set exact width and height in pixels
- ✓ Lock aspect ratio — provide one dimension, the other is computed
- ✓ LANCZOS resampling for highest quality downscaling
- ✓ UUID output filename
- ✓ Before/after comparison viewer
- ✓ Works on all supported image formats
- ✓ Returns resized image immediately
- ✓ Handles both enlargement and reduction

HOW IT WORKS

Pillow `Image.open()` loads the image. If `keep_aspect=true` and only one dimension is supplied, the other is computed proportionally using the original aspect ratio. `Image.resize((width, height), Image.LANCZOS)` applies the resize. LANCZOS is the highest quality resampling filter in Pillow, using a sinc function to minimise aliasing.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	Image file	Required	Image to resize
<code>width</code>	Integer	Conditional	Target width in pixels
<code>height</code>	Integer	Conditional	Target height in pixels
<code>keep_aspect</code>	Boolean	Optional	Lock aspect ratio (default: true)

OUTPUT

Resized image at the specified dimensions.

LIBRARIES

Pillow (PIL)

Flask

USAGE TIPS

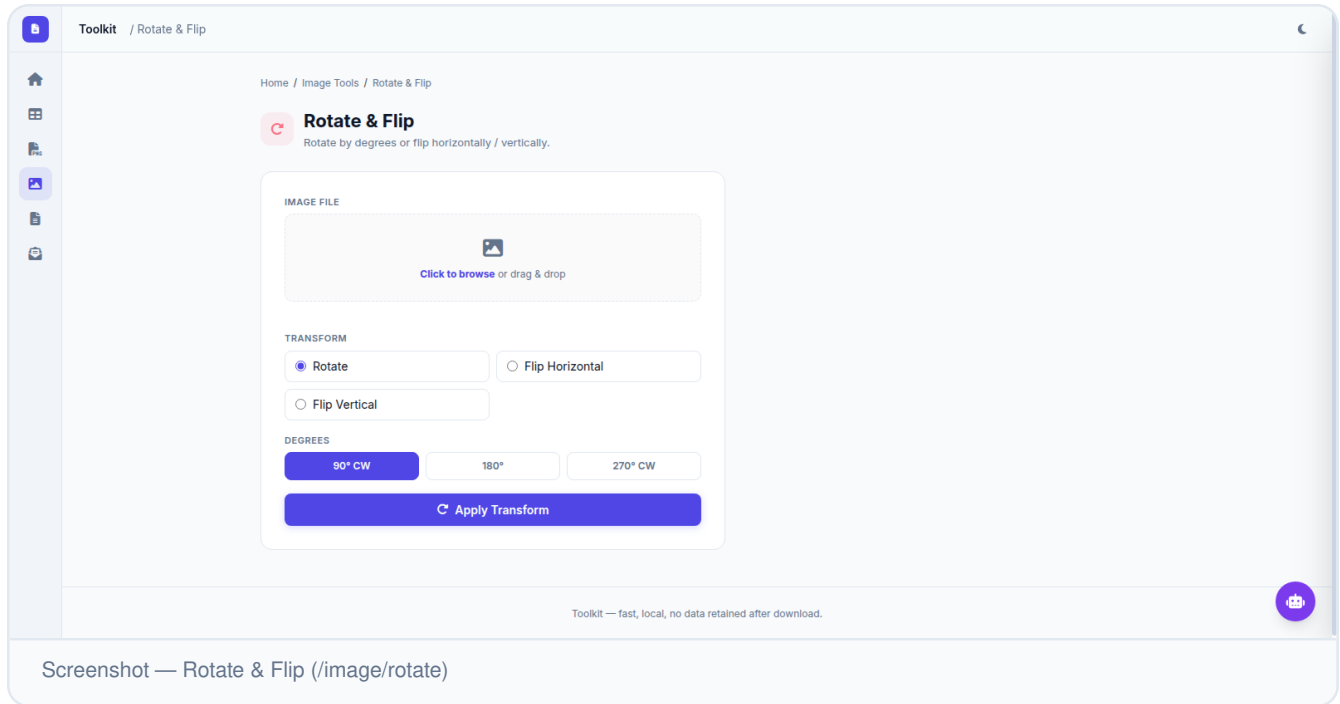
1. Always lock aspect ratio unless you intentionally want to stretch the image.
2. LANCZOS is slower but produces sharper results than bilinear or nearest-neighbour.
3. Enlarging an image beyond 2× its original size will cause visible blurring.
4. Resize to a standard size (1920×1080, 1280×720) for web or presentation use.
5. Combine with Compress after resizing to minimise file size further.

EDITOR'S NOTE

Scaling images to precise dimensions is a common requirement across publishing, web optimization, and asset pipeline workflows. This tool accepts target width and height values and applies LANCZOS resampling — a high-quality downsampling algorithm that minimizes aliasing artifacts by using a sinc-based kernel — to produce sharp, accurate output. Aspect-ratio locking is enabled by default, preventing unintended distortion when only one dimension is specified; both dimensions must be explicitly provided to override this behavior. Practitioners should note that LANCZOS resampling carries a higher computational cost than bicubic or bilinear alternatives, making it better suited for offline processing than real-time pipelines handling high image volumes.

Rotate & Flip

Rotates an image 90, 180, or 270 degrees clockwise, or flips it horizontally or vertically.



Screenshot — Rotate & Flip (/image/rotate)

KEY FEATURES

- ✓ Rotate 90°, 180°, or 270° clockwise
- ✓ Flip horizontally (mirror left-right)
- ✓ Flip vertically (mirror top-bottom)
- ✓ `expand=True` preserves full image content after rotation
- ✓ UUID output filename
- ✓ Before/after comparison viewer
- ✓ Works on all supported image formats
- ✓ Returns transformed image immediately

HOW IT WORKS

For rotation: `Image.rotate(degrees, expand=True)` rotates the image. `expand=True` grows the canvas to accommodate the rotated image so no corners are cropped. For flips: `Image.transpose(Image.FLIP_LEFT_RIGHT)` or `Image.transpose(Image.FLIP_TOP_BOTTOM)`. Both operations preserve all pixel data. Saves to UUID path.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	Image file	Required	Image to transform
<code>degrees</code>	90 180 270	Conditional	Rotation angle (for rotate direction)
<code>direction</code>	rotate flip_h flip_v	Required	Operation type

OUTPUT

Rotated or flipped image.

LIBRARIES

Pillow (PIL)

Flask

USAGE TIPS

1. Use 90° rotation to fix images taken in landscape that display as portrait.

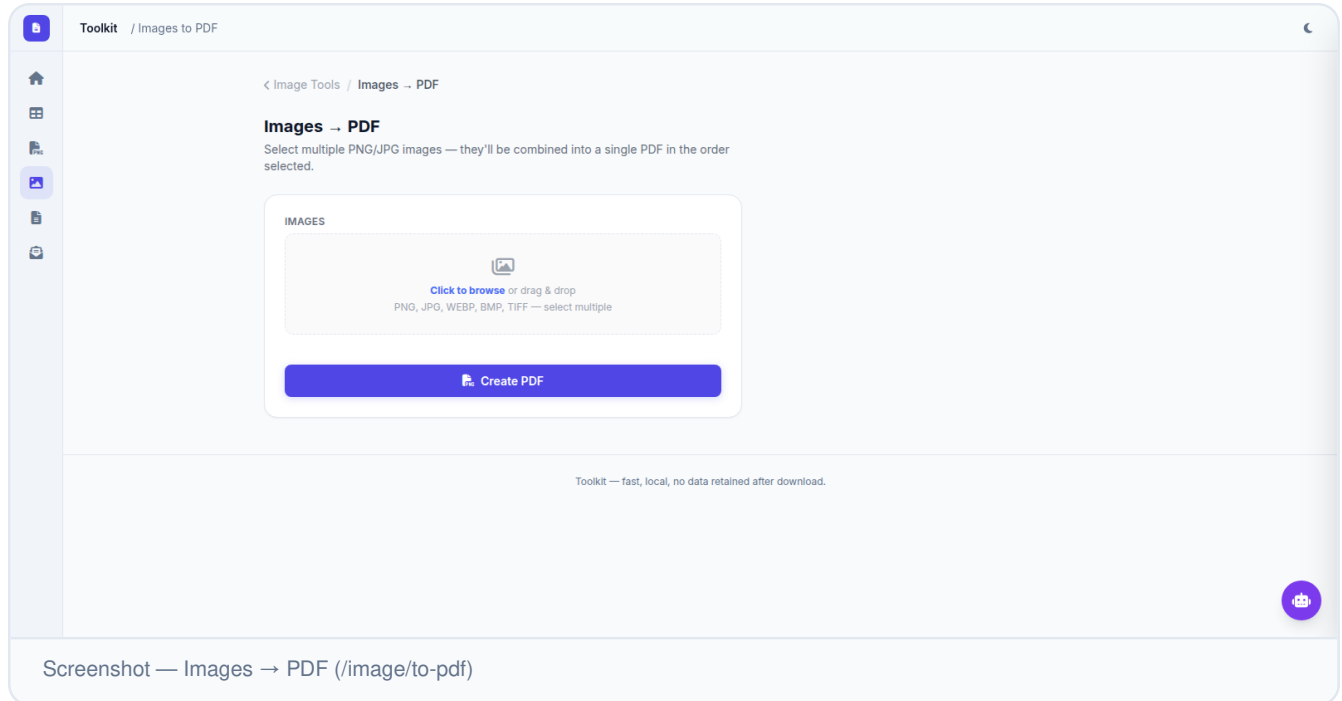
2. Horizontal flip creates a mirror image — useful for watermark removal workarounds.
3. 180° rotation fixes upside-down scanned documents.
4. `expand=True` means the rotated image fills the frame completely.
5. Combine multiple rotations by running the tool twice if needed.

EDITOR'S NOTE

Orientation correction is one of the most common image preprocessing steps, and this tool addresses it directly by exposing Pillow's transform pipeline for rotations at 90, 180, and 270 degrees alongside horizontal and vertical flips. Each operation is lossless relative to the pixel data — no resampling or interpolation is applied, since all supported transforms map pixels to exact integer coordinates. A key design consideration is EXIF orientation handling: Pillow does not automatically apply embedded EXIF rotation metadata, so callers should strip or normalize that tag after transformation to avoid double-rotation in downstream viewers. Flips and 90-degree rotations are equally inexpensive; 270 degrees is internally equivalent to three successive 90-degree rotations.

Images → PDF

Combines multiple images into a single multi-page PDF document. Each image becomes one page at its original dimensions.



KEY FEATURES

- ✓ Combine any number of images into one PDF
- ✓ Each image becomes one page at its original pixel dimensions
- ✓ Images processed in upload order
- ✓ RGBA/palette images converted to RGB for PDF compatibility
- ✓ UUID output filename
- ✓ Returns multi-page PDF immediately
- ✓ Accepts PNG, JPG, WEBP, BMP, TIFF
- ✓ No compression — images embedded at full quality

HOW IT WORKS

Pillow opens each image and converts to RGB mode (required for PDF compatibility). The first image is saved as a PDF using `Image.save(path, "PDF")`. Subsequent images are appended using the `save_all` parameter: `first_img.save(path, save_all=True, append_images=rest)`. Each page matches the pixel dimensions of its source image.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>files[]</code>	Image files	Required	Two or more images to combine into a PDF

OUTPUT

Multi-page PDF with one image per page.

LIBRARIES

Pillow (PIL)

Flask

USAGE TIPS

1. Upload images in the order you want them to appear in the PDF.
2. Images with different dimensions result in pages of different sizes.

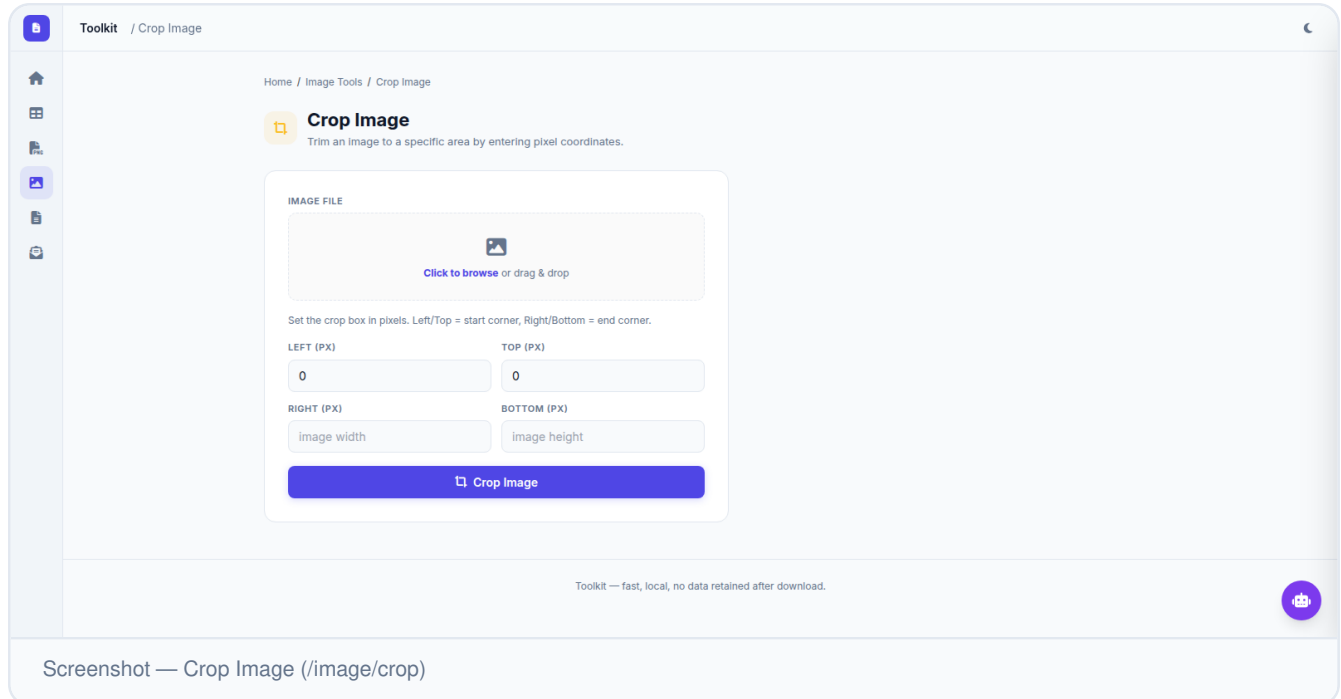
3. For a consistent page size, resize all images first using the Resize tool.
4. WEBP images are converted to RGB before embedding.
5. Use Compress PDF after conversion to reduce the output file size.

EDITOR'S NOTE

Converting a collection of images into a single multi-page PDF is a common archival and distribution task that this tool handles through Pillow's ``save_all`` parameter, which instructs the library to treat each subsequent frame or image as an additional page rather than overwriting the output file. Input images are accepted in any format Pillow supports — JPEG, PNG, BMP, TIFF, and others — though all images are normalized to RGB before writing, since PDF output via Pillow does not support transparency or palette modes. Users should ensure input images share a consistent resolution or aspect ratio when visual uniformity matters, as Pillow does not automatically resize pages to match one another. File order determines page order, so callers are responsible for sorting inputs before invocation.

Crop Image

Trims an image to a rectangular area defined by pixel coordinates. Coordinates are clamped to the image bounds automatically.



Screenshot — Crop Image (/image/crop)

KEY FEATURES

- ✓ Crop to exact pixel coordinates
- ✓ Left/Top define the top-left corner of the crop box
- ✓ Right/Bottom define the bottom-right corner
- ✓ Coordinates clamped to image bounds — no out-of-range errors
- ✓ JPEG output converts RGBA to RGB automatically
- ✓ UUID output filename
- ✓ Before/after comparison viewer
- ✓ Works on all supported image formats

HOW IT WORKS

Pillow `Image.open()` gets the image and its dimensions. Coordinates are clamped: `left=max(0,left)`, `top=max(0,top)`, `right=min(w,right)`, `bottom=min(h,bottom)`. `Image.crop((left, top, right, bottom))` creates the cropped image. If output is JPEG and source has alpha, `convert("RGB")` is applied. Saves to UUID path.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	Image file	Required	Image to crop
<code>left</code>	Integer	Required	Left edge in pixels from left border
<code>top</code>	Integer	Required	Top edge in pixels from top border
<code>right</code>	Integer	Required	Right edge in pixels from left border
<code>bottom</code>	Integer	Required	Bottom edge in pixels from top border

OUTPUT

Cropped image containing only the specified rectangular region.

LIBRARIES

Pillow (PIL)

Flask

USAGE TIPS

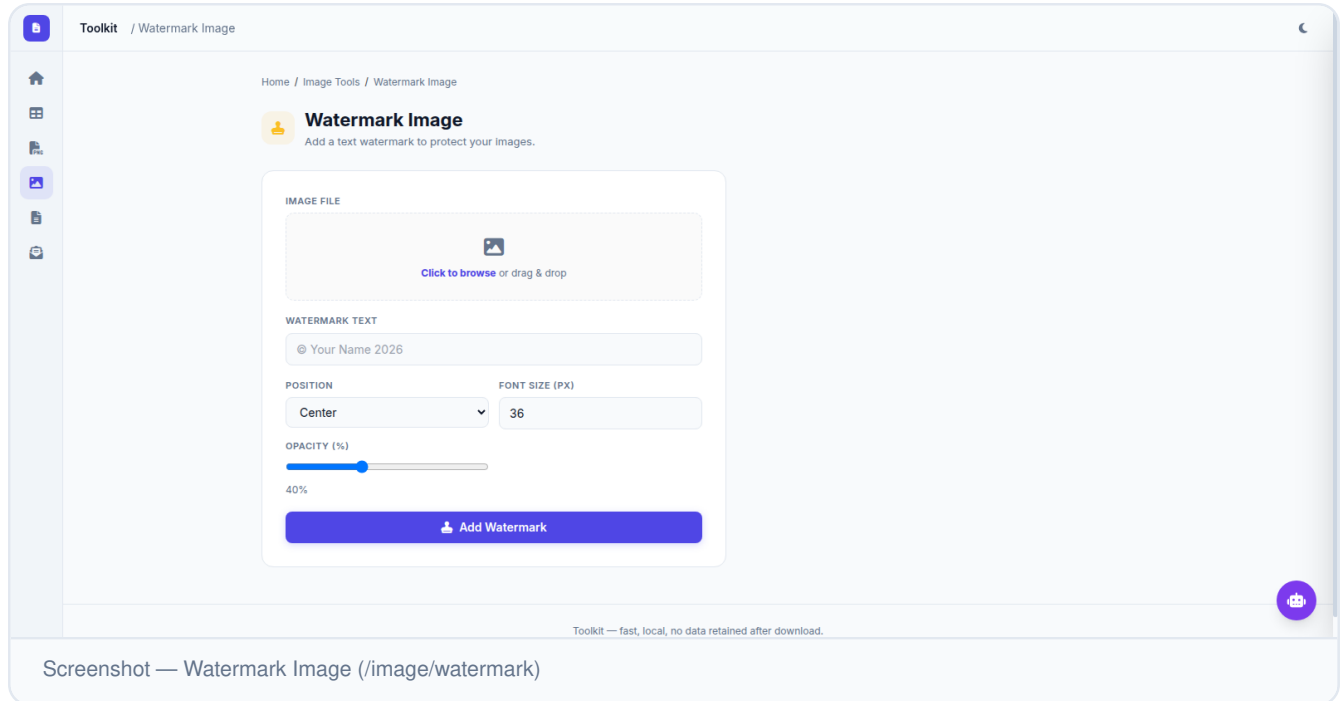
1. Use an image editor to find pixel coordinates if you're not sure of the exact values.
2. left must be less than right; top must be less than bottom.
3. Cropping does not reduce resolution — only the canvas size changes.
4. For a 1920×1080 image, a centre crop would be (480, 270, 1440, 810).
5. Combine with Resize after cropping to standardise output dimensions.

EDITOR'S NOTE

Precise region extraction is the primary use case for the Crop Image tool, which accepts a pixel-coordinate bounding box defined as (left, top, right, bottom) and delegates the actual crop operation to Pillow's `Image.crop`` method. A notable design decision is auto-clamping: coordinates exceeding image dimensions are silently clamped to valid bounds rather than raising an exception, which prevents pipeline failures when bounding boxes are computed programmatically from approximate or scaled sources. Practitioners should be aware that Pillow's crop does not validate that left is less than right or top is less than bottom; inverted coordinates produce a zero-dimension or nonsensical result, so callers must normalize bounding boxes before invocation.

Watermark Image

Overlays a semi-transparent text watermark on an image with configurable position, opacity, and font size.



Screenshot — Watermark Image (/image/watermark)

KEY FEATURES

- ✓ Configurable watermark text
- ✓ Five positions: center, bottom-right, bottom-left, top-right, top-left
- ✓ Opacity control from 5% to 100%
- ✓ Font size control from 10px to 200px
- ✓ White text with dark shadow for visibility on any background
- ✓ RGBA layer compositing for true transparency
- ✓ UUID output filename
- ✓ Returns watermarked image immediately

HOW IT WORKS

Image converted to RGBA. A fully transparent layer (`Image.new("RGBA", size, (0,0,0,0))`) is created. `ImageDraw` renders the text twice: white with specified alpha for the main text, dark with half alpha offset by 1px for the shadow. `Image.alpha_composite(base, txt_layer)` merges the layers. Final image converted to RGB for JPEG output.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>file</code>	Image file	Required	Image to watermark
<code>text</code>	String	Required	Watermark text
<code>position</code>	String	Optional	Placement position (default: center)
<code>opacity</code>	1-100	Optional	Text opacity percentage (default: 40)
<code>font_size</code>	Integer	Optional	Font size in pixels (default: 36)

OUTPUT

Watermarked image with semi-transparent text overlay.

LIBRARIES

Pillow

ImageDraw

ImageFont

Flask

USAGE TIPS

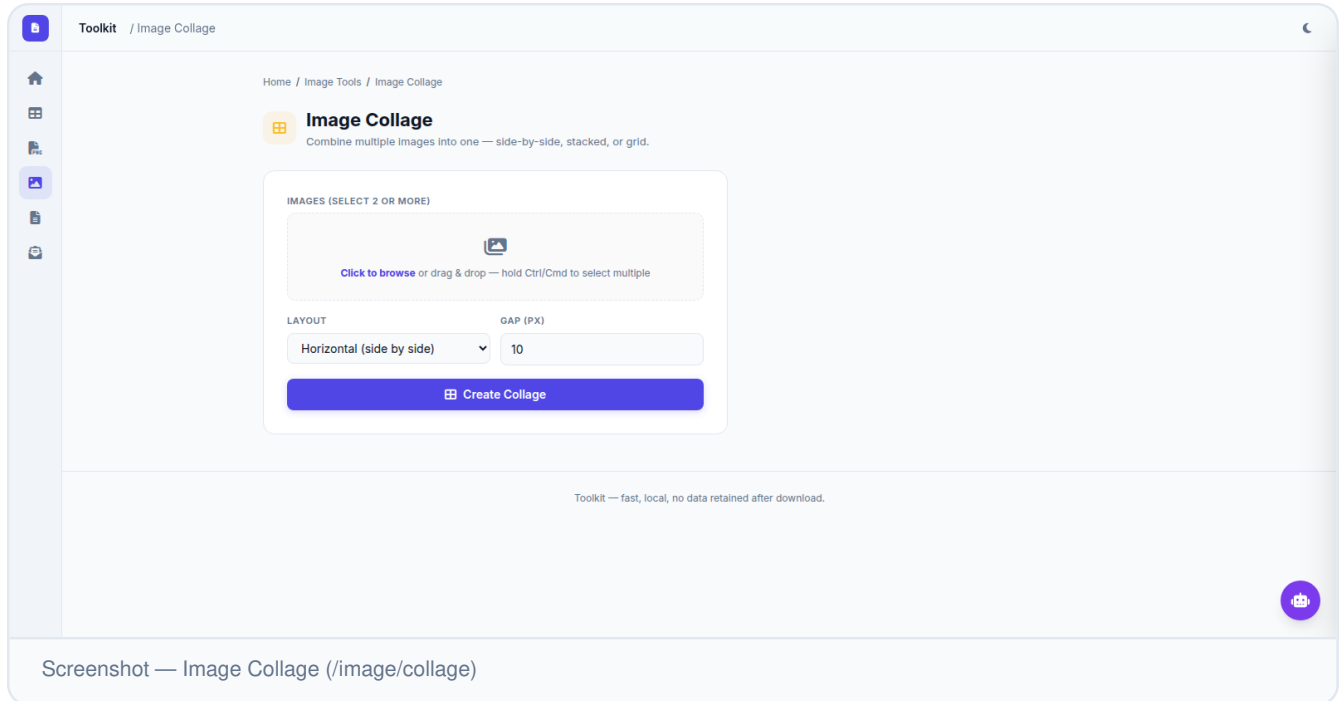
1. 40% opacity gives a visible but non-obtrusive watermark.
2. Use bottom-right for subtle branding, center for strong protection.
3. Larger images need larger font sizes to keep the watermark proportional.
4. The shadow effect ensures the text is readable on both light and dark backgrounds.
5. LiberationSans-Bold is used if available, falling back to Pillow's default font.

EDITOR'S NOTE

Overlaying ownership or branding information onto images without degrading the original content is a common requirement for asset pipelines and publishing workflows. This tool accomplishes that by compositing a semi-transparent RGBA text layer directly onto the source image using Pillow's alpha compositing engine, with a drop shadow added to ensure legibility across varied backgrounds. The RGBA approach preserves visual fidelity while keeping the watermark non-destructive in perception, though the operation itself modifies pixel data permanently — the original file is not preserved unless the caller manages copies explicitly. Practitioners should select font size and opacity relative to the target image dimensions; defaults optimized for large images will obscure detail on thumbnails.

Image Collage

Combines 2 or more images into a single composite image in horizontal, vertical, or 2-column grid layout.



Screenshot — Image Collage (/image/collage)

KEY FEATURES

- ✓ Horizontal layout: all images resized to same height, placed side-by-side
- ✓ Vertical layout: all images resized to same width, stacked top-to-bottom
- ✓ Grid layout: 2-column grid with uniform cell size
- ✓ Configurable gap between images in pixels
- ✓ White background fills any gaps
- ✓ Output saved as JPEG at quality 90
- ✓ UUID output filename
- ✓ Returns collage immediately

HOW IT WORKS

Pillow opens all images as RGB. Horizontal: resize all to max height preserving aspect, compute total width, paste sequentially with gap. Vertical: resize all to max width, compute total height, paste with gap. Grid: find max cell size, resize all to that size, paste at $(col*(cell_w+gap), row*(cell_h+gap))$. Final canvas saved as JPEG.

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>files[]</code>	Image files	Required	2 or more images to combine
<code>layout</code>	horizontal vertical grid	Optional	Layout type (default: horizontal)
<code>gap</code>	Integer	Optional	Gap between images in pixels (default: 10)

OUTPUT

Single composite JPEG image combining all inputs in the chosen layout.

LIBRARIES

Pillow (PIL)

Flask

USAGE TIPS

1. Grid layout works best with an even number of images.
2. Use `gap=0` for a seamless borderless collage.
3. Images with similar aspect ratios produce the most visually consistent results.
4. Horizontal layout is best for panoramic or wide compositions.
5. Resize all images to the same dimensions before collaging for a uniform grid.

EDITOR'S NOTE

Combining multiple images into a single composite, the Image Collage tool accepts two or more source files and arranges them across three layout modes — horizontal, vertical, or grid — using direct canvas pasting via the Pillow imaging library. Layout selection is the primary design decision: horizontal and vertical modes preserve original image dimensions along the primary axis, while grid mode distributes images into a balanced rectangular arrangement, padding incomplete rows as needed. Practitioners should ensure source images share compatible color modes (RGB or RGBA) before processing, as mixed modes can produce unexpected blending artifacts during canvas composition. Output dimensions are computed from the aggregate of all inputs, so large source files proportionally increase memory consumption and write time.

Resume & Documents

Resume Maker

Builds a professional resume with a live preview across 6 templates. Supports all standard resume sections including certifications, projects, languages, and awards.

The screenshot shows the Resume Maker web application. On the left, there's a 'TEMPLATES' sidebar with four options: Modern 1, Modern 2, Modern 3, and Modern 4. The main area is divided into two sections: 'Contact Information' and 'Professional Summary'. The 'Contact Information' section has fields for Full Name (Jane Smith), Email (jane@example.com), Phone (+1 555 000 0000), Location (New York, NY), LinkedIn (linkedin.com/in/jane), and Website / Portfolio (janesmith.dev). The 'Professional Summary' section has a text area for a brief overview. Below this is the 'Work Experience' section with a '+ Add' button. On the right, there's a 'PREVIEW' section showing a live preview of the resume. At the top right, there are buttons for 'Refresh Preview' and 'Download PDF'. The bottom of the screenshot shows the URL 'Screenshot — Resume Maker (/resume/)'.

KEY FEATURES

- ✓ Sections: Contact, Summary, Experience, Education, Skills, Certifications, Projects, Languages, Awards
- ✓ Dynamic add/remove blocks for all multi-entry sections
- ✓ Star-rating picker for skill proficiency levels (1–5)
- ✓ 6 professional templates: Modern 1–6
- ✓ Live preview updates as you type (700ms debounce)
- ✓ Preview rendered in scaled iframe via srcdoc injection
- ✓ PDF download via WeasyPrint
- ✓ Dark mode synced with Toolkit localStorage key

HOW IT WORKS

ResumeData dataclass holds all resume sections. `_parse_form()` reads the flat multidict,

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>name/email/ phone/ location</code>	String	Required	Contact information
<code>summary</code>	Text	Optional	Professional summary paragraph
<code>company/ role/ exp_start/ exp_end/ bullets</code>	Repeated	Optional	Work experience entries (add multiple)
<code>cert_name/ cert_issuer/ cert_date</code>	Repeated	Optional	Certification entries

OUTPUT

HTML preview at `/resume/preview` or PDF download at `/resume/download`.

grouping repeated fields (`company[]`, `role[]`, `bullets[]`) by array index into `WorkExperience` objects. `render_resume_html()` calls `Jinja2` `render_template(template_file, **data_dict)`. Live preview POSTs to `/resume/preview` and injects the HTML response into an `iframe` `srcdoc`. `WeasyPrint.HTML(string=html).write_pdf()` generates the downloadable PDF.

LIBRARIES

WeasyPrint

Flask

Jinja2

Python dataclasses

USAGE TIPS

1. Fill in Contact first — it appears in the preview immediately.
2. Add skills using the star picker for visual proficiency indicators.
3. Switch between the 6 templates using the left sidebar — preview updates instantly.
4. Certifications and Projects sections are optional — leave them empty to omit from the PDF.
5. Download the PDF and open in a viewer before sharing to confirm formatting.

EDITOR'S NOTE

Resume Maker is a full-featured document authoring tool that guides users through nine structured sections — including summary, experience, education, skills, and certifications — with real-time preview rendering as content is entered. Six layout templates cover common professional formats, from minimalist single-column designs to structured two-column variants. PDF export is handled by WeasyPrint, which renders HTML/CSS to print-accurate output; this means template styling must use WeasyPrint-compatible CSS properties, and complex CSS features such as flexbox gap or certain pseudo-elements may require fallbacks. Operators running this tool in containerized environments should verify WeasyPrint's system-level font and library dependencies are present in the base image to avoid silent rendering failures.

Cover Letter Maker

Creates a tailored cover letter with live preview across 3 professional templates. PDF download powered by WeasyPrint.

The screenshot shows the 'Cover Letter Maker' web application. On the left, there's a 'TEMPLATES' sidebar with three options: 'Classic', 'Modern', and 'Minimal'. The main area is divided into two sections: 'Your Contact Information' and 'Letter Details'. The 'Your Contact Information' section includes fields for Full Name, Email, Phone, Location, and LinkedIn. The 'Letter Details' section includes fields for Date, Company, Job Title, and Hiring Manager (optional). Below these is an 'Opening Paragraph' field. On the right, there's a 'PREVIEW' section showing a live preview of the cover letter. The preview includes the user's name, contact information, and the body of the letter. At the top right of the preview, there are buttons for 'Refresh Preview' and 'Download PDF'. The bottom of the screenshot shows the URL 'Screenshot — Cover Letter Maker (/cover-letter/)'.

KEY FEATURES

- ✓ Sections: Contact, Letter Details, Opening, Key Experience, Why This Company, Closing
- ✓ All body sections are free-form text areas
- ✓ 3 templates: Classic (serif), Modern (navy header), Minimal (clean sans)
- ✓ Live preview updates with 600ms debounce
- ✓ Hiring manager field optional — defaults to "Hiring Manager"
- ✓ PDF download via WeasyPrint
- ✓ Dark mode synced with Toolkit localStorage key
- ✓ "← Toolkit" navigation preserves toolkit context

HOW IT WORKS

CoverLetterData dataclass stores all fields. `_parse_form()` performs a simple flat form read. `_to_ctx()` converts to a Jinja2-friendly dict. Templates render the four body paragraphs (opening, body_1, body_2, closing) with the

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
name/email/ phone/location	String	Required	Your contact information
date/company/ job_title	String	Required	Letter details
hiring_manager	String	Optional	Recipient name (default: Hiring Manager)
opening/ body_1/body_2/ closing	Text	Optional	Letter body — four separate paragraphs

OUTPUT

HTML preview at `/cover-letter/preview` or PDF download at `/cover-letter/download`.

appropriate salutation using the `hiring_manager` field. WeasyPrint generates the PDF.

LIBRARIES

WeasyPrint

Flask

Jinja2

Python dataclasses

USAGE TIPS

1. Opening: introduce yourself and state the specific role you're applying for.
2. Key Experience (`body_1`): lead with a concrete measurable achievement.
3. Why This Company (`body_2`): show genuine research about the company.
4. Closing: include a clear call to action and express enthusiasm.
5. Use the Modern template for tech roles and Classic for traditional industries.

EDITOR'S NOTE

Cover Letter Maker generates polished, print-ready cover letters from structured user input, organizing content across four discrete body sections to enforce clear narrative flow: introduction, relevant experience, motivation, and closing call to action. Three built-in templates — formal, modern, and minimal — address the most common application contexts without requiring custom CSS knowledge. A live preview panel renders changes in real time via the browser, reducing iteration cycles before committing to PDF export. PDF generation relies on WeasyPrint, which requires a system-level installation and correct font paths; deployments should validate WeasyPrint availability at startup rather than at export time to surface configuration errors early.

Invoice Generator

Generates professional invoices with dynamic line items, automatic tax and total calculation, and PDF download. Two templates available.

The screenshot displays the 'Invoice Maker' web application. On the left, there are two template options: 'Classic' and 'Modern'. The main form is divided into sections: 'Invoice Details' with fields for Invoice Number, Currency, Invoice Date, and Due Date; 'Line Items' with a table for Description, QTY, and UNIT PRICE; 'Tax & Notes' with fields for Tax Rate, Discount, and a free-text field for Notes/Payment Terms. On the right, a 'PREVIEW' section shows a sample invoice for 'Your Business' (Acme Corp) with a table of line items and a total due of \$4400.00. The interface includes a 'Refresh Preview' button and a 'Download PDF' button.

Screenshot — Invoice Generator (/invoice/)

KEY FEATURES

- ✓ Dynamic line items: description, quantity, unit price per row
- ✓ Automatic subtotal, tax amount, and grand total computation
- ✓ Configurable tax rate as a percentage
- ✓ Sender and client information sections
- ✓ Invoice number, date, and due date fields
- ✓ Notes/payment terms free-text field
- ✓ 2 templates: Classic (traditional) and Modern (dark header)
- ✓ PDF download via WeasyPrint

HOW IT WORKS

InvoiceData dataclass contains LinelItem objects. Computed properties on InvoiceData: $\text{subtotal} = \sum(\text{item.qty} * \text{item.unit_price for item in items})$, $\text{tax_amount} = \text{subtotal} * \text{tax_rate} / 100$, $\text{grand_total} = \text{subtotal} + \text{tax_amount}$. `_parse_form()` zips `item_desc[]`,

INPUTS

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>sender_name/</code> <code>email/phone/</code> <code>address</code>	String	Required	Your business information
<code>client_name/</code> <code>email/address</code>	String	Required	Client information
<code>invoice_number/</code> <code>date/due_date</code>	String	Required	Invoice metadata
<code>item_desc[]/</code> <code>item_qty[]/</code> <code>item_price[]</code>	Repeated arrays	Required	Line items — add as many as needed
<code>tax_rate</code>	Float	Optional	Tax percentage (e.g. 10 for 10%)

OUTPUT

HTML preview at `/invoice/preview` or PDF download at `/invoice/download`.

item_qty[], item_price[] arrays into Lineltem objects. _to_ctx() pre-computes all totals for the Jinja2 template. WeasyPrint generates the PDF.

LIBRARIES

WeasyPrint

Flask

Jinja2

Python dataclasses

USAGE TIPS

1. Add line items first — the preview updates with running totals immediately.
2. Tax rate of 0 hides the tax row in the output.
3. Use the invoice number field consistently (e.g. INV-0001, INV-0002) for record-keeping.
4. The due date appears prominently — set it to 30 days after the invoice date by convention.
5. Use the Notes field for bank transfer details or payment instructions.

EDITOR'S NOTE

Generating invoices programmatically requires more than static templates — line-item variability, tax calculations, and branded output must all compose reliably. This tool addresses those requirements through a dynamic line-item model where subtotals, taxes, and grand totals are computed server-side on each render, eliminating client-side drift. Two built-in templates cover standard business and itemized service layouts; WeasyPrint handles PDF generation, meaning output fidelity depends on a correctly installed WeasyPrint environment with its native library dependencies (libpango, libcairo). Operators should verify those system libraries are present before deployment. For custom branding, extend the existing Jinja2 template rather than forking the computation logic to keep tax rules centralized.

